

Computer Organization and Architecture

Unit – 3

The Memory System

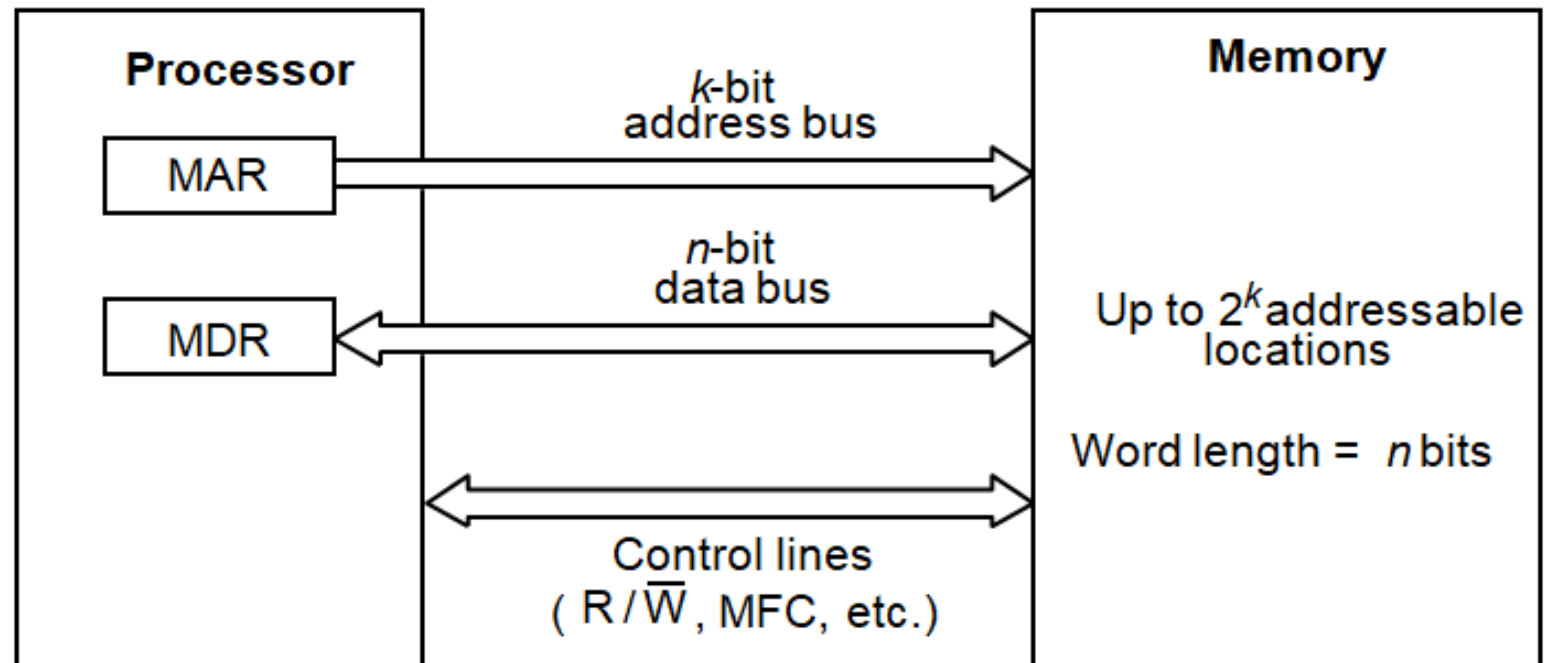
Some basic concepts

- Programs and the Data
- Fast , Large and Inexpensive Memory
- Maximum size of memory that can be used in any computer is determined by addressing Scheme

Address	Memory Locations
16 Bit	$2^{16} = 64 \text{ K}$
32 Bit	$2^{32} = 4\text{G (Giga)}$
40 Bit	$2^{40} = \text{IT (Tera)}$

A	B
0	0
0	1
1	0
1	1

A	B	C
0	0	0
0	0	1
0	1	0



CPU-Main Memory Connection

Some basic concepts

- **Memory Access Time:** It is the time that elapses between
 - initiation of an operation &
 - completion of that operation.
- **Memory Cycle Time:** It is the minimum time delay that required between the initiation of the two successive memory-operations.

RAM (Random Access Memory)

- In RAM, any location can be accessed for a Read/Write-operation in fixed amount of time,

Cache Memory

- It is a small, fast memory that is inserted between
 - larger slower main-memory and
 - processor.
- It holds the currently active segments of a program and their data.

Some basic concepts

- If MAR is k-bits long then
 - memory may contain upto 2^k addressable-locations
 - If MDR is n-bits long, then
 - n-bits of data are transferred between the memory and processor.
 - The data-transfer takes place over the processor-bus (Figure 8.1).
 - The processor-bus has
 - 1) Address-Line
 - 2) Data-line &
 - 3) Control-Line (R/W", MFC – Memory Function Completed).
 - The Control-Line is used for coordinating data-transfer.
 - The processor reads the data from the memory by
 - loading the address of the required memory-location into MAR and
 - setting the R/W" line to 1.
 - The memory responds by
 - placing the data from the addressed-location onto the data-lines and
 - confirms this action by asserting MFC signal.
 - Upon receipt of MFC signal, the processor loads the data from the data-lines into MDR.
 - The processor writes the data into the memory-location by
 - loading the address of this location into MAR &
 - setting the R/W" line to 0.
-

Some basic concepts

Introduction

- Memory is one of the most important functional units of a computer.
 - Used to store both instructions and data.
 - Stores as bits (0's and 1's), usually organized in terms of bytes.
- How are the data stored in memory accessed?
 - Every memory location has a unique address.
 - A memory is said to be byte addressable if every byte of data has a unique address.
 - Some memory systems are word addressable also (every addressed locations consists of multiple bytes, say, 32 bits or 4 bytes).

Classification of Memory Systems

a) Volatile versus Non-volatile:

- A *volatile* memory system is one where the stored data is lost when the power is switched off.
 - Examples: CMOS static memory, CMOS dynamic memory.
 - Dynamic memory in addition requires periodic refreshing.
- A *non-volatile* memory system is one where the stored data is retained even when the power is switched off.
 - Examples: Read-only memory, Magnetic disk, CDROM/DVD, Flash memory, Resistive memory.

b) Random-access versus Direct/Sequential access:

- A memory is said to be *random-access* when the read/write time is independent of the memory location being accessed.
 - Examples: CMOS memory (RAM and ROM).
- A memory is said to be *sequential access* when the stored data can only be accessed sequentially in a particular order.
 - Examples: Magnetic tape, Punched paper tape.
- A memory is said to be *direct or semi-random access* when part of the access is sequential and part is random.
 - Example: Magnetic disk.
 - We can directly go to a track after which access will be sequential.

c) Read-only versus Random-access:

- *Read-only Memory* (ROM) is one where data once stored in permanent or semi-permanent.
 - Data written (programmed) during manufacture or in the laboratory.
 - Examples: ROM, PROM, EPROM, EEPROM.
- *Random Access Memory* (RAM) is one where data access time is the same independent of the location (address).
 - Used in main / cache memory systems.
 - Example: Static RAM (SRAM) → data once written are retained as long as power is on.
 - Example: Dynamic RAM (DRAM) → requires periodic refreshing even when power is on (data stored as charge on tiny capacitors).

• Memory can be classified as follows:

1) RAM which can be further classified as follows:

- i) Static RAM
- ii) Dynamic RAM (DRAM) which can be further classified as synchronous & asynchronous DRAM.

2) ROM which can be further classified as follows:

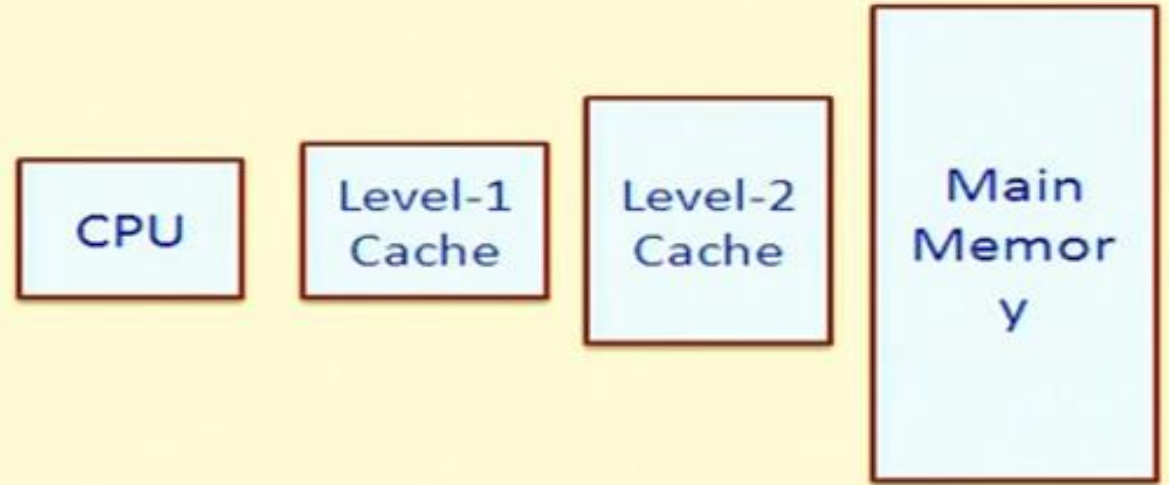
- i) PROM
- ii) EPROM
- iii) EEPROM &
- iv) Flash Memory which can be further classified as Flash Cards & Flash Drives.

Access Time, Latency and Bandwidth

- Terminologies used to measure speed of the memory system.
 - a) **Memory Access Time**: Time between initiation of an operation (Read or Write) and completion of that operation.
 - b) **Latency**: Initial delay from the initiation of an operation to the time the first data is available.
 - c) **Bandwidth**: Maximum speed of data transfer in bytes per second.
- In modern memory organizations, every read request reads a block of words into some high-speed registers (LATENCY), from where data are supplied to the processor one by one (ACCESS TIME).
- Some important questions?
 - How to make the memory system work faster?
 - How to increase the data transfer rate between CPU and memory?
 - How to address the ever increasing storage needs of applications?
- Some possible solutions:
 - **Cache Memory**: to increase the effective speed of the memory system.
 - **Virtual Memory**: to increase the effective size of the memory system.

What is Cache Memory?

- A fast memory (possibly organized in several levels) that sits between processor and main memory.
- Faster than main memory and relatively small.
- Frequently accessed data and instructions are stored here.
- Cache memory makes use of the fast SRAM technology.

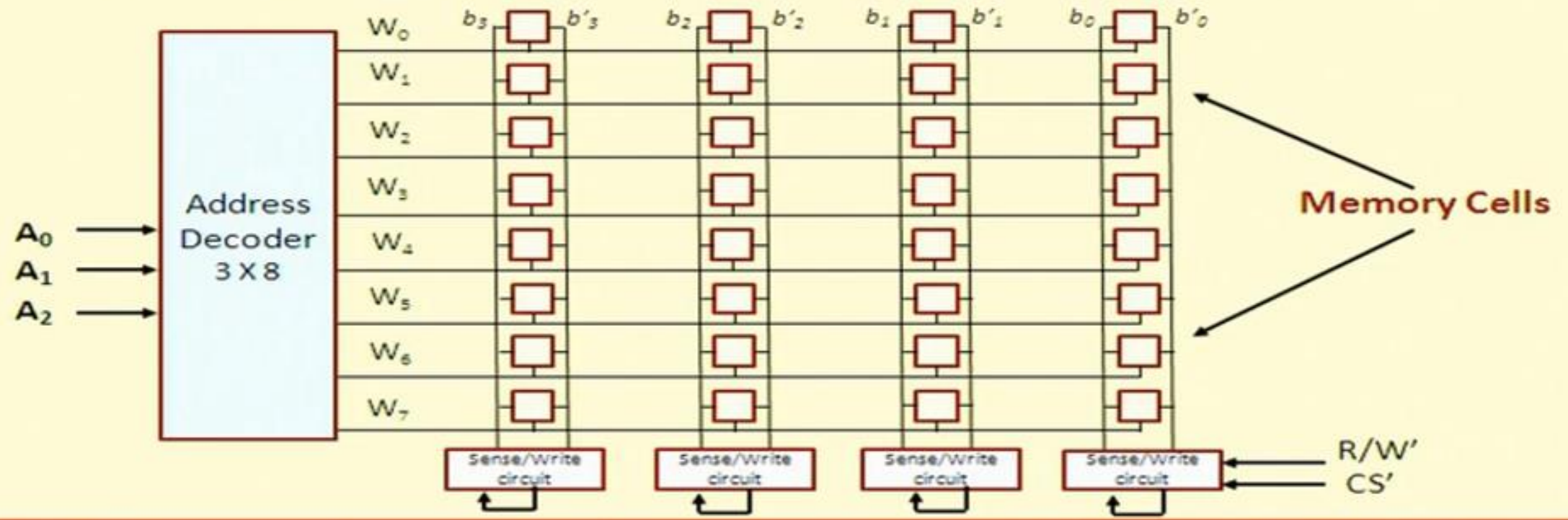


How a Memory Chip Looks Like?

- Memory cells are organized in the form of an array.
- Every memory cell holds one bit of data.
- Present-day VLSI technology allows one to pack billions of bits per chip.
- A memory module used in computers typically contains several such chips.



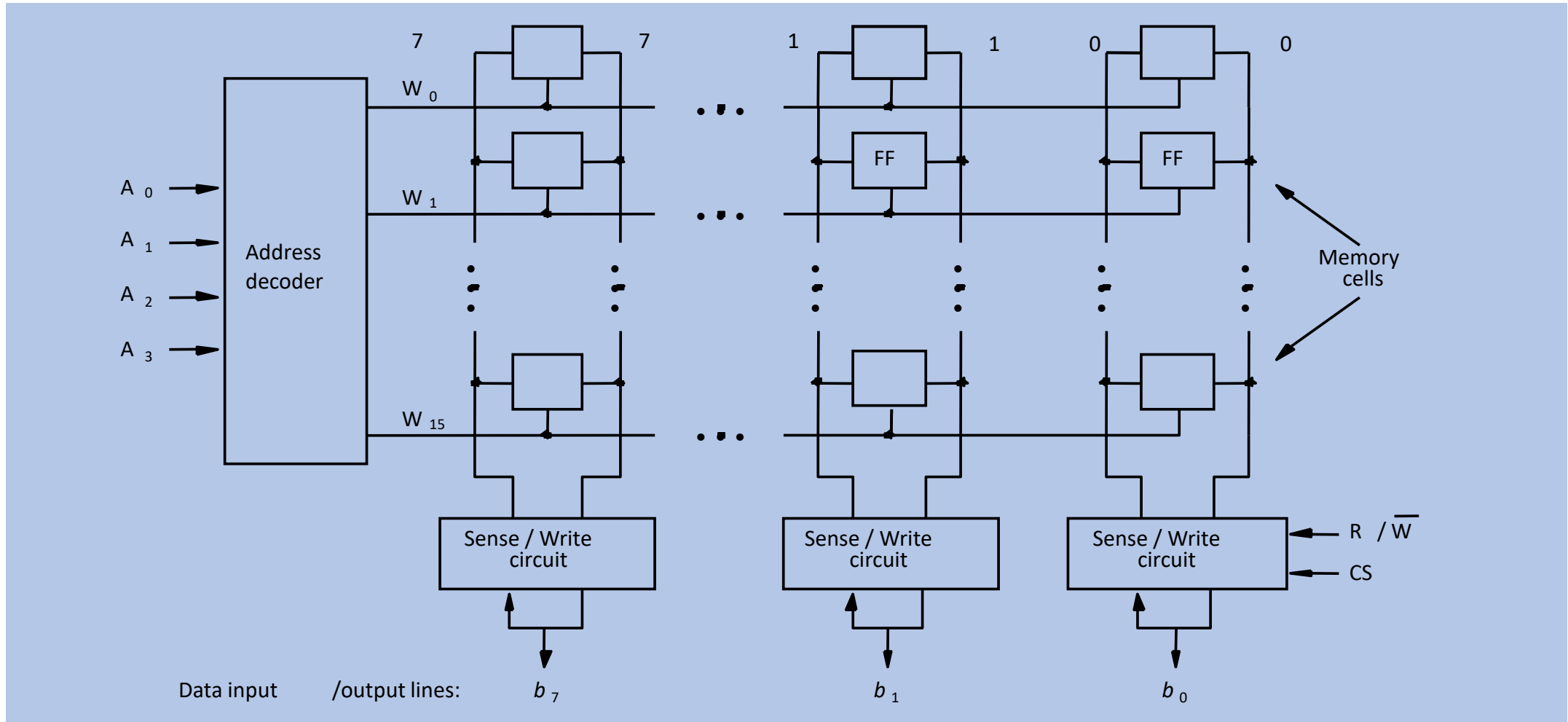
Organization of Cells in an 8x4 Memory Chip



- A 32-bit memory chip organized as 8×4 is shown.
- Every row of the cell array constitutes a memory word.
- A 3×8 decoder is required to access any one of the 8 rows.
- The rows of the cells are connected to the word lines.
- Individual cells are connected to two bit lines.
 - Bit b and its complement b' .
 - Required for reading and writing.
- Cells in each column are connected to a sense/write circuit by the two bit lines.
- Other than address and data lines, there are two control lines: R/W' and CS' (Chip Select).
 - CS is required to select one single chip in a multi-chip memory system.

Internal organization of memory chips (Contd.,)

(16 * 8 organization – 16 words of 8 bits each)



SEMI CONDUCTOR RAM MEMORIES

INTERNAL ORGANIZATION OF MEMORY-CHIPS

- Memory-cells are organized in the form of array (Figure 8.2).
- Each cell is capable of storing 1-bit of information.
- Each row of cells forms a memory-word.
- All cells of a row are connected to a common line called as **Word-Line**.
- The cells in each column are connected to **Sense/Write** circuit by 2-bit-lines.
- The Sense/Write circuits are connected to data-input or output lines of the chip.
- During a write-operation, the sense/write circuit
 - receive input information &
 - store input info in the cells of the selected word.
- The data-input and data-output of each Sense/Write circuit are connected to a single bidirectional data-line.
- Data-line can be connected to a data-bus of the computer.
- Following 2 control lines are also used:
 - 1) **R/W'** → Specifies the required operation.
 - 2) **CS'** → Chip Select input selects a given chip in the multi-chip memory-system.

External Connection Requirements

The 8 x 4 memory requires the following external connections:

- Address decoder of size: 3 x 8
 - 3 external connections for address.
- Data output : 4-bit
 - 4 external connections for data.
- 2 external connections for R/W' and CS'.
- 2 external connections for power supply and ground.
- Total of $3 + 4 + 2 + 2 = 11$.

What About a 256 X 16 Memory?

Here the total number of external connections are estimated as follows.

- Address decoder size: 8×256
 - 8 external connections for address.
- Data output : 16-bit
 - 16 external connections for data.
- 2 external connections for R/W' and CS'.
- 2 external connections for power supply and ground.
- Total of $8 + 16 + 2 + 2 = 28$.

Introduction

- Broadly two types of semiconductor memory systems:
 - a) Static Random Access Memory (SRAM)
 - b) Dynamic Random Access Memory (DRAM)
 - i. Asynchronous DRAM
 - ii. Synchronous DRAM
- Vary in terms of speed, density, volatility properties, and cost.
 - Present-day main memory systems are built using DRAM.
 - Cache memory systems are built using SRAM.

Static Random Access Memory (SRAM)

- SRAM consists of circuits which can store the data as long as power is applied.
- It is a type of semiconductor memory that uses bistable latching circuitry (flip-flop) to store each bit.
- SRAM memory arrays can be arranged in rows and columns of memory cells.
 - Called *word line* and *bit line*.

SRAM technology:

- Can be built using 4 or 6 MOS transistors.
- Modern SRAM chips in the market uses 6-transistor implementations for CMOS compatibility.
- Widely used in small-scale systems like microcontrollers and embedded systems.
- Also used to implement cache memories in computer systems.
 - To be discussed later.



Some basic concepts

STATIC RAM (OR MEMORY)

- Memories consist of circuits capable of retaining their state as long as power is applied are known.
- Two inverters are cross connected to form a latch (Figure 8.4).
- The latch is connected to 2-bit-lines by transistors T_1 and T_2 .
- The transistors act as switches that can be opened/closed under the control of the word-line.
- When the word-line is at ground level, the transistors are turned off and the latch retain its state.

Read Operation

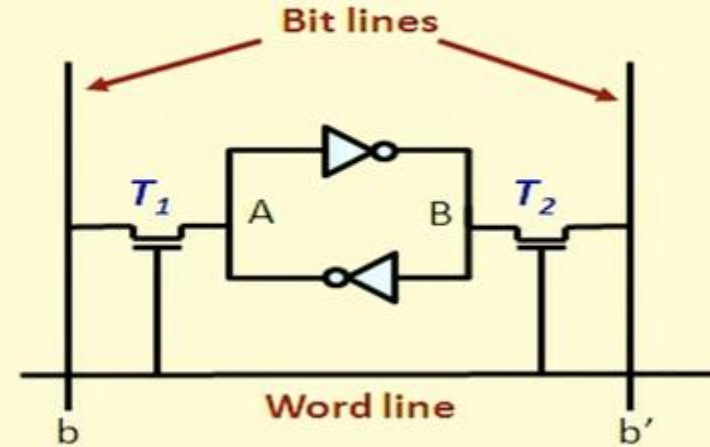
- To read the state of the cell, the word-line is activated to close switches T_1 and T_2 .
- If the cell is in state 1, the signal on bit-line b is high and the signal on the bit-line b'' is low.
- Thus, b and b'' are complement of each other.
- Sense/Write circuit
 - monitors the state of b & b'' and
 - sets the output accordingly.

Write Operation

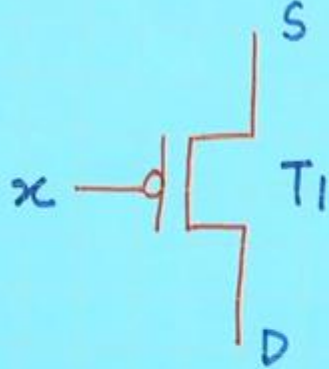
- The state of the cell is set by
 - placing the appropriate value on bit-line b and its complement on b'' and
 - then activating the word-line. This forces the cell into the corresponding state.
- The required signal on the bit-lines is generated by Sense/Write circuit.

A 1-bit SRAM Cell

- Two inverters are cross connected to form a latch.
- The latch is connected to two bit lines with transistors T_1 and T_2 .
- Transistors behave like switches that can be opened (OFF) or closed (ON) under the control of the word line.
- To retain the state of the latch, the word line can be grounded which makes the transistors off.

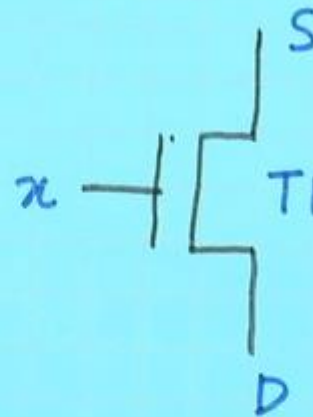


PMOS TRANSISTOR



When $x = 0$
There is a conducting path
from S to D.

NMOS TRANSISTOR

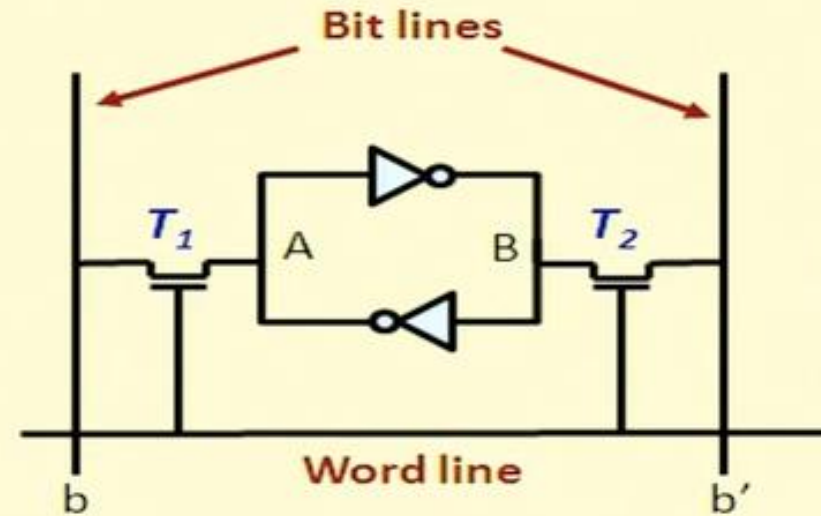


When $x = 1$
There is a conducting
path from S to D.

© CET
I.I.T. KGP

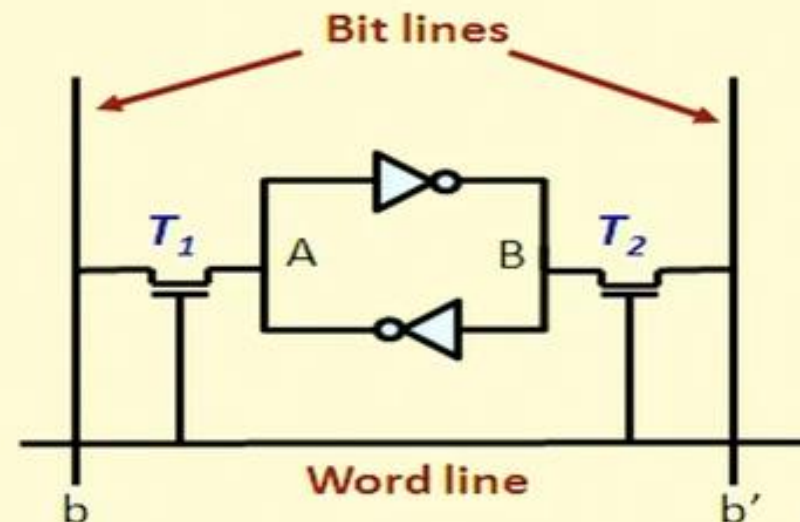
READ Operation in SRAM

- To read the content of the cell, the word line is activated (= 1) to make the transistors T_1 and T_2 on.
- The value stored in latch is available on bit line b and its complement on b' .
- Sense/write circuits connected to the bit lines monitor the states of b and b' .



WRITE Operation in SRAM

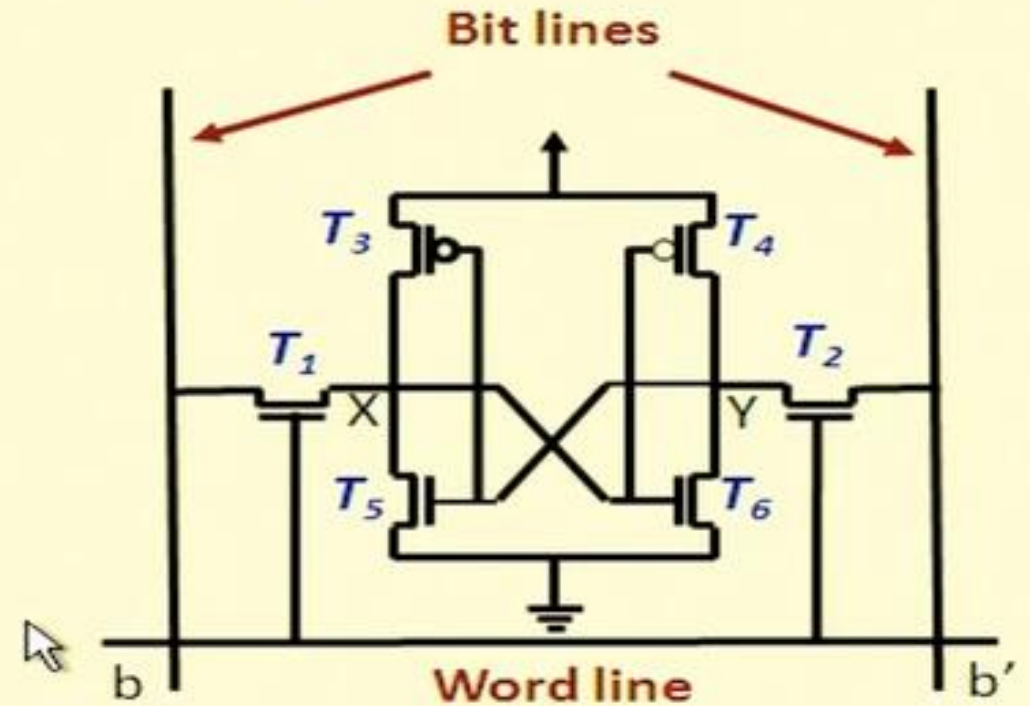
- **To write 1:** The bit line b is set with 1 and bit line b' is set with 0. Then the word line is activated and the data is written to the latch.
- **To write 0:** The bit line b is set with 0 and bit line b' is set with 1. Then the word line is activated and the data is written to the latch.
- The required signals (either 1 or 0) are generated by the sense/write circuit.

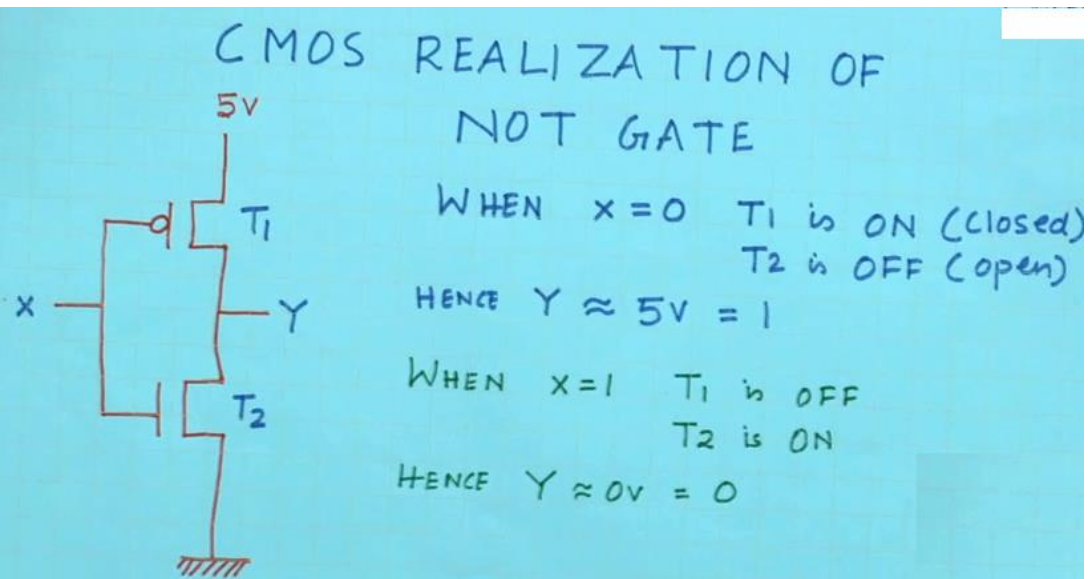


6 Transistor Static CMOS RAM Cell

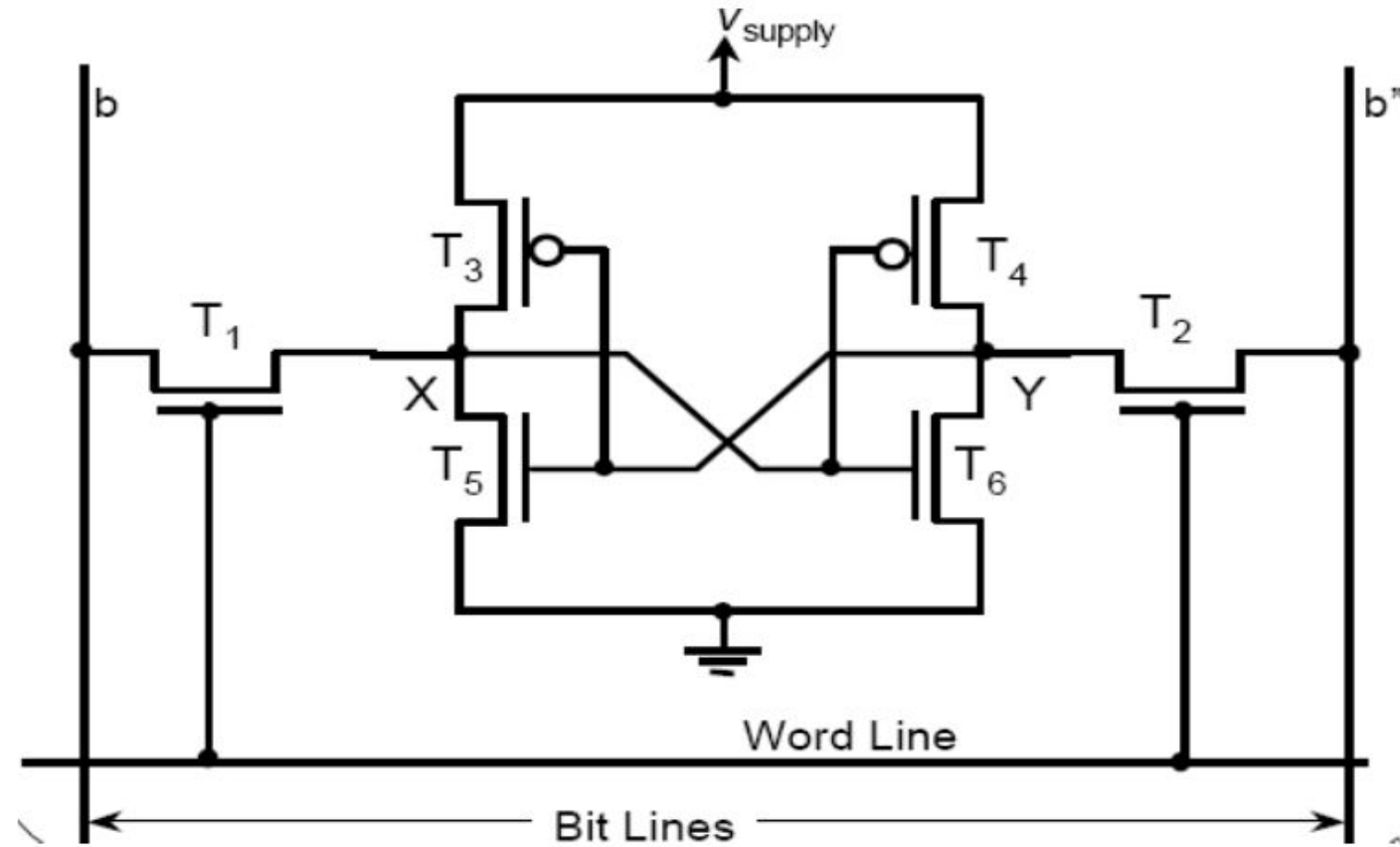
6-Transistor Static Memory cell

- 1-bit SRAM cell with 6-transistors are used in modern-day SRAM implementations.
- Transistors (T_3 & T_5) and (T_4 & T_6) form the CMOS inverters in the latch.
- The data can be read or written in the same way as explained.



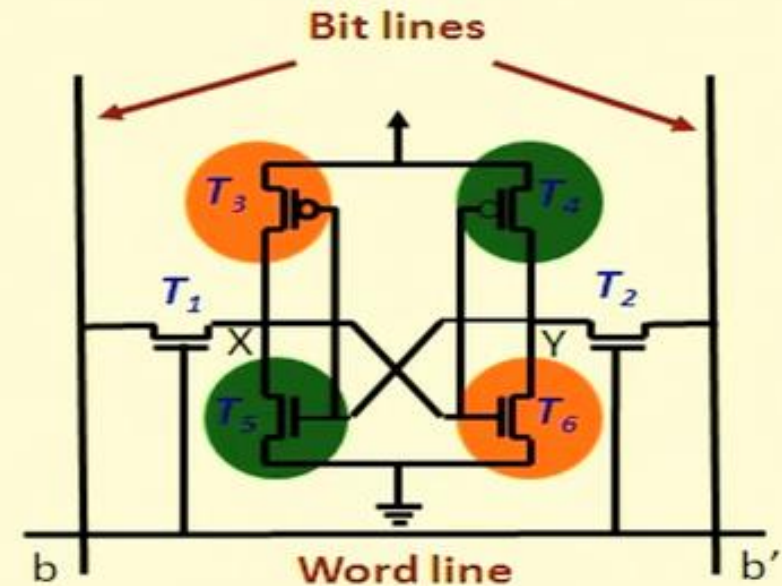


A CMOS Memory Cell



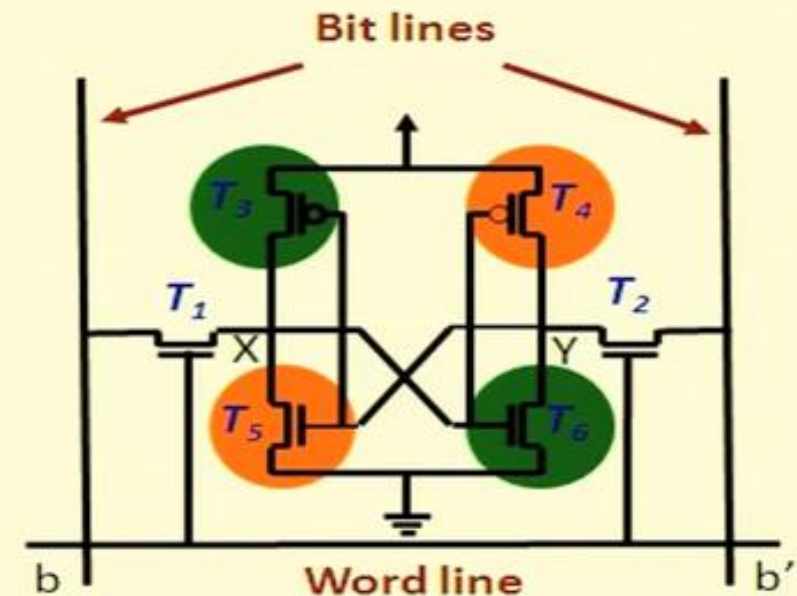
In State 0

- In state 0 the voltage at X is low and the voltage at Y is high.
- When the voltage at X is low, transistors (T_4 & T_5) are on while (T_3 & T_6) are off.
- When word line is activated, T_1 and T_2 are turned on and the bit lines b will have 0 and b' will have 1.



In State 1

- In state 1 the voltage at X is high and the voltage at Y is low.
- When the voltage at X is high, transistors (T_3 & T_6) are on while (T_4 & T_5) are off.
- When word line is activated, T_1 and T_2 are turned on and the bit lines b will have 1 and b' will have 0.



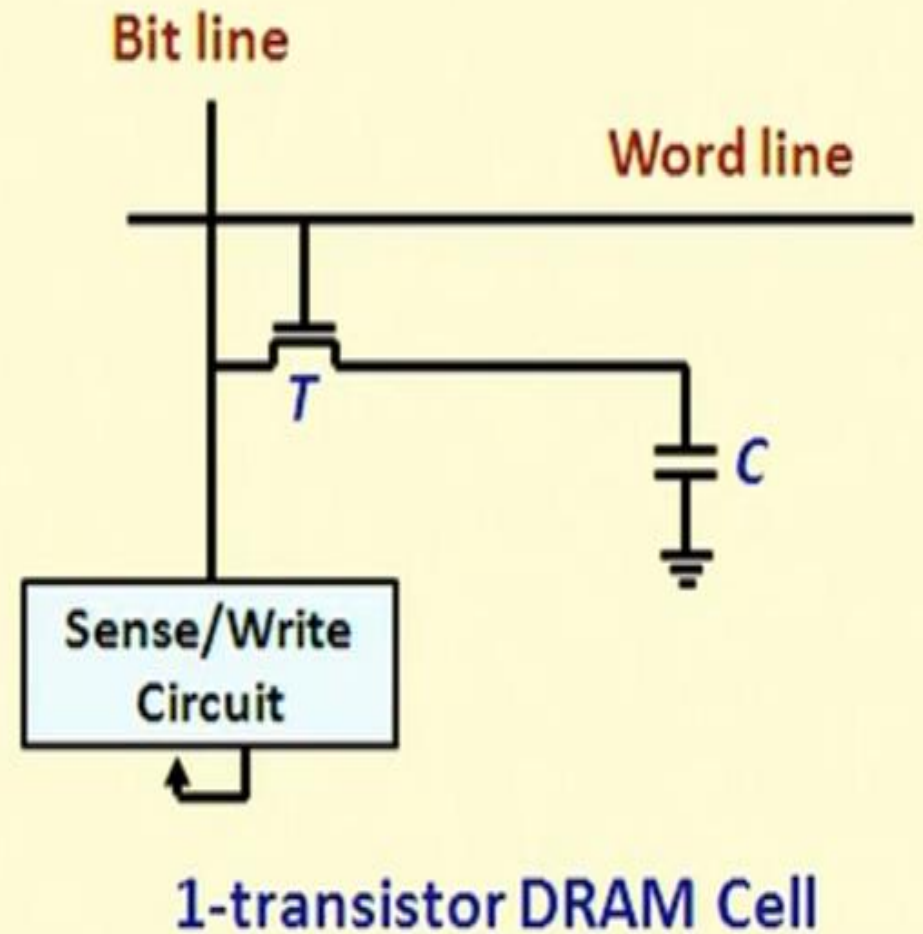
Features of SRAM

- Moderate / High power consumption.
 - Current flows in the cells only when the cell is accessed.
 - Because of latch operation, power consumption is higher than DRAM.
- Simplicity – refresh circuitry is not needed.
 - Volatile :: continuous power supply is required.
- Fast operation.
 - Access time is very fast; fast memories (cache) are built using SRAM.
- High cost.
 - 6 transistors per cell.
- Limited capacity.
 - Not economical to manufacture high-capacity SRAM chips.

DRAM – Slower & Less expensive

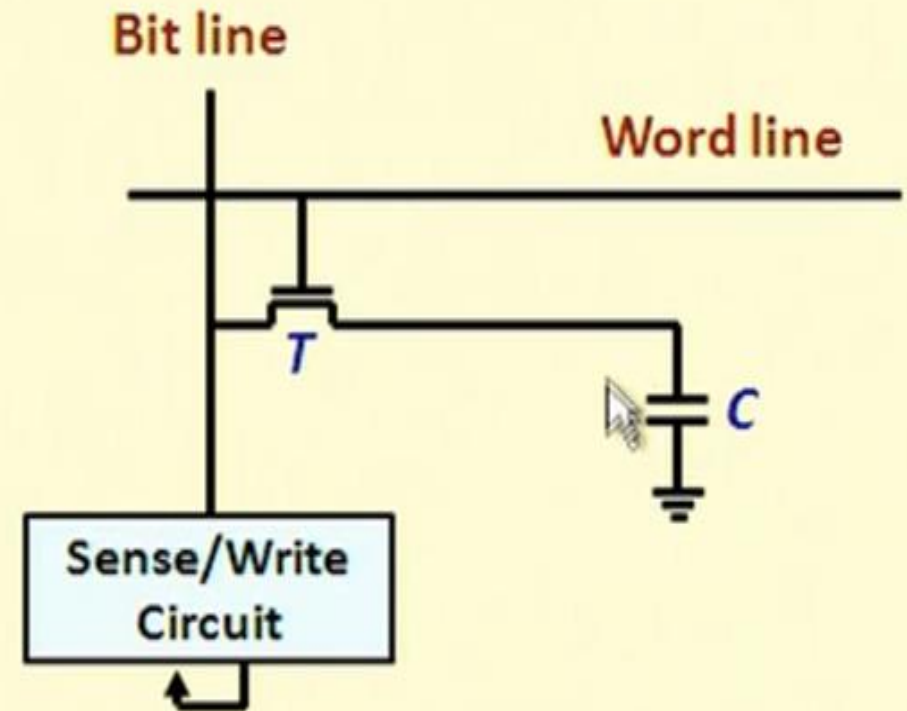
Dynamic Random Access Memory (DRAM)

- Dynamic RAM do not retain its state even if power supply is on.
 - Data stored in the form of charge stored on a capacitor.
- Requires periodic refresh.
 - The charge stored cannot be retained over long time (due to leakage).
- Less expensive than SRAM.
 - Requires less hardware (one transistor and one capacitor per cell).
- Address lines are multiplexed.



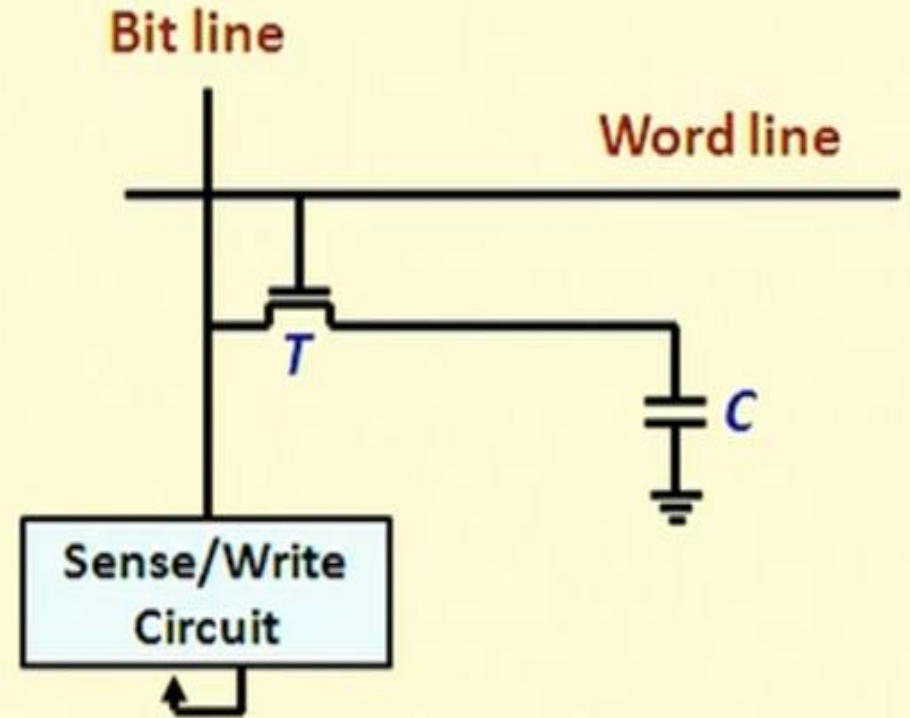
READ Operation in DRAM

- The transistor of the particular cell is turned on by activating the word line.
- A sense amplifier connected to bit line senses the charge stored in the capacitor.
- If the charge is above threshold, the bit line is maintained at high voltage, which represents logic **1**.
- If the charge is below threshold, the bit line is grounded, which represent logic **0**.



WRITE Operation in DRAM

- The transistor of the particular cell is turned on by activating the word line.
- Depending on the value to be written (*0* or *1*), an appropriate voltage is applied to the bit line.
- The capacitor gets charged to the required voltage state.
- Refreshing of the capacitor requires periodic READ-WRITE cycles (every few msec).



Types of DRAM

a) Asynchronous DRAM (ADRAM)

- Timing of the memory device is handled asynchronously.
- A special memory controller circuit generates the signals asynchronously.
- DRAM chips produced between the early 1970s to mid-1990s used *asynchronous* DRAM.

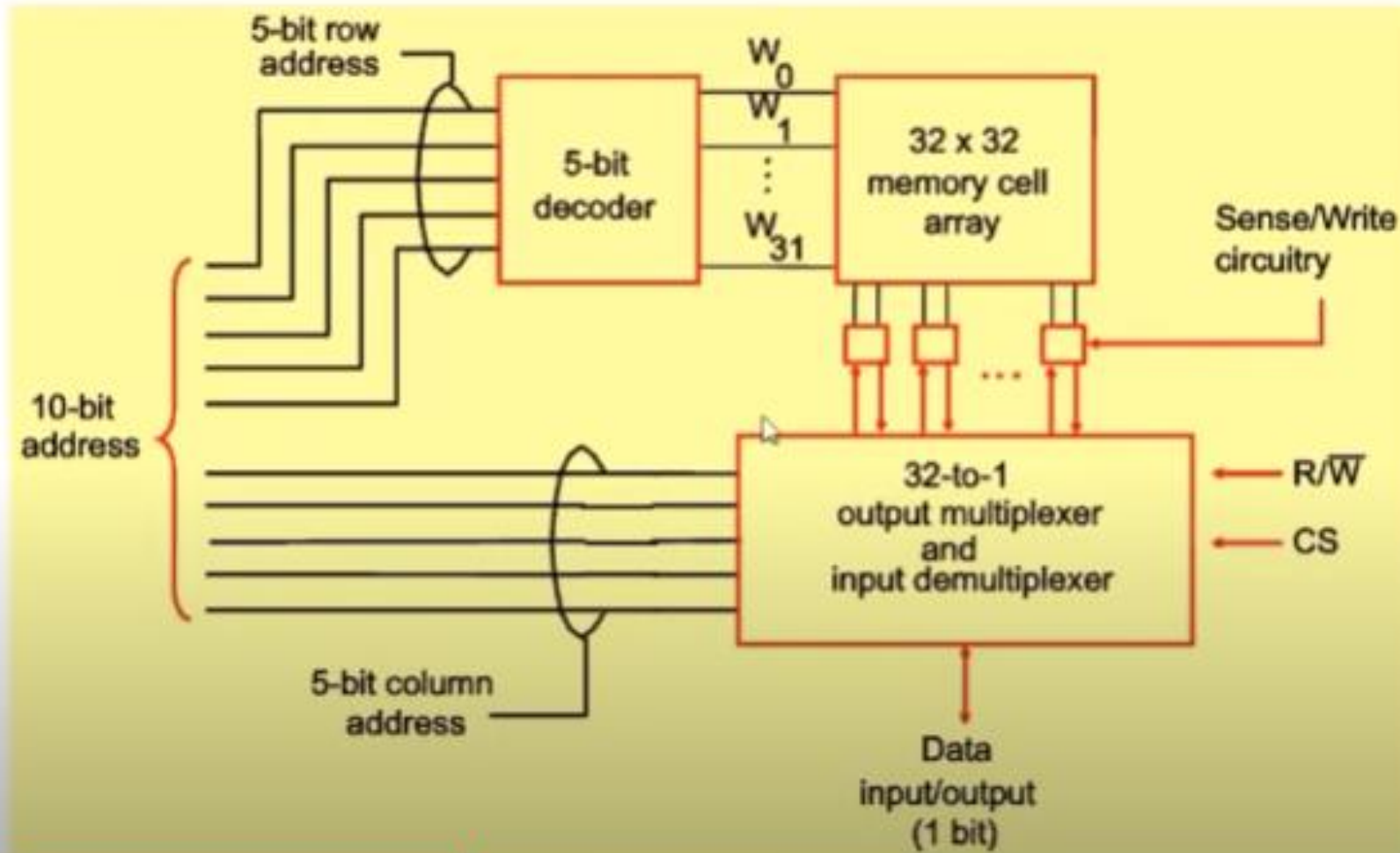
b) Synchronous DRAM (SDRAM)

- Memory operations are synchronized by a clock.
- Concept of SDRAM came in the 1970s.
- Commercially made available only in 1993 by Samsung.
- By 2000 SDRAM replaced almost all types of DRAMs in the market.
- Performance of SDRAM is much higher compared to all other existing DRAM.

Organization of a 1K x 1 memory chip

$$1K = 2^{10} = 1024$$

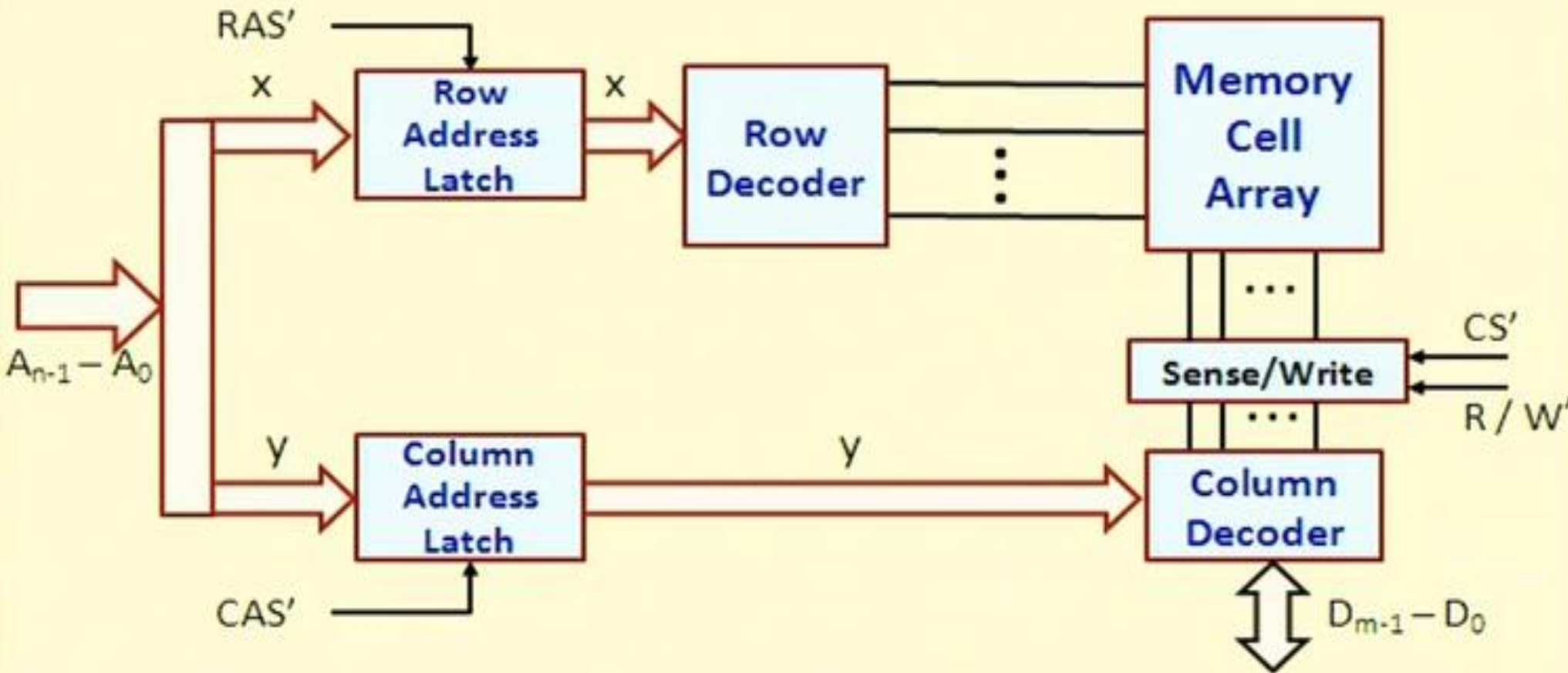
$$32 * 32 = 1024$$



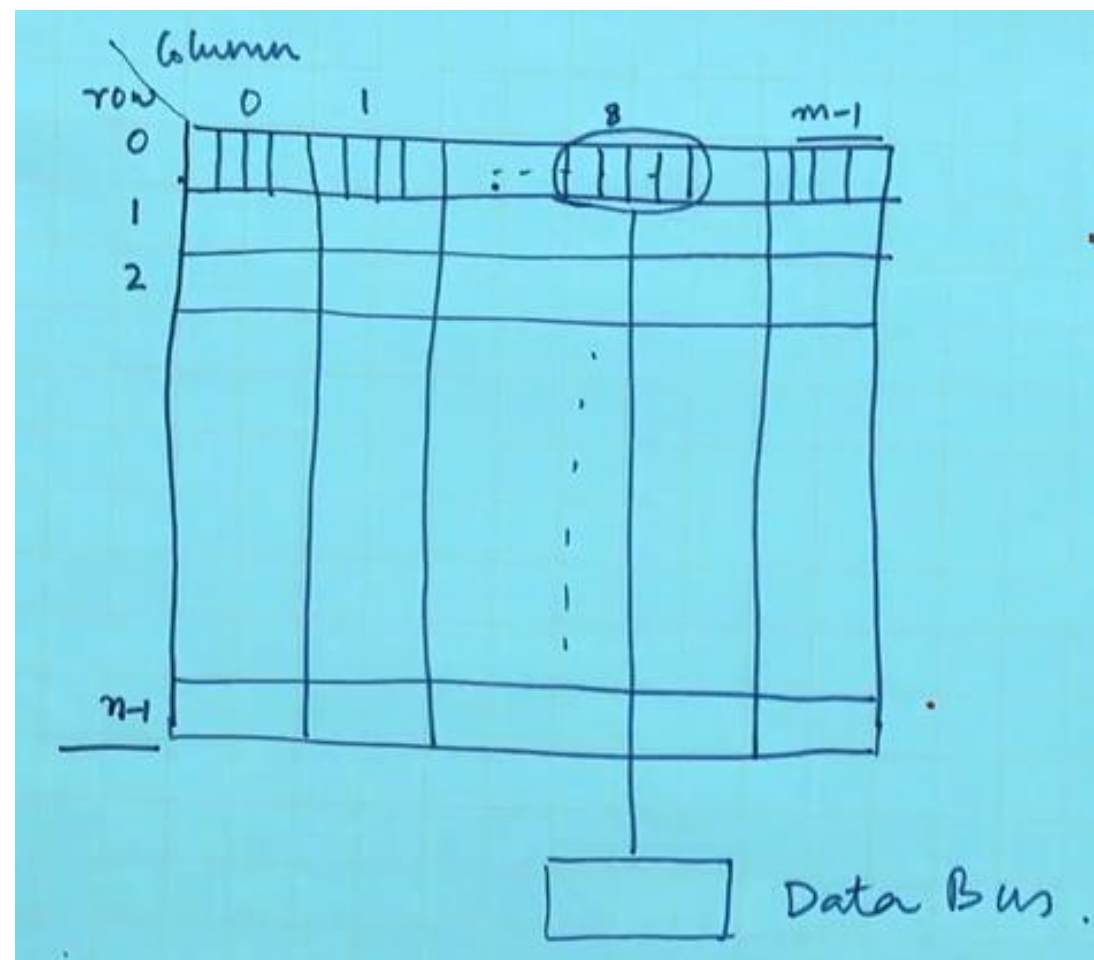
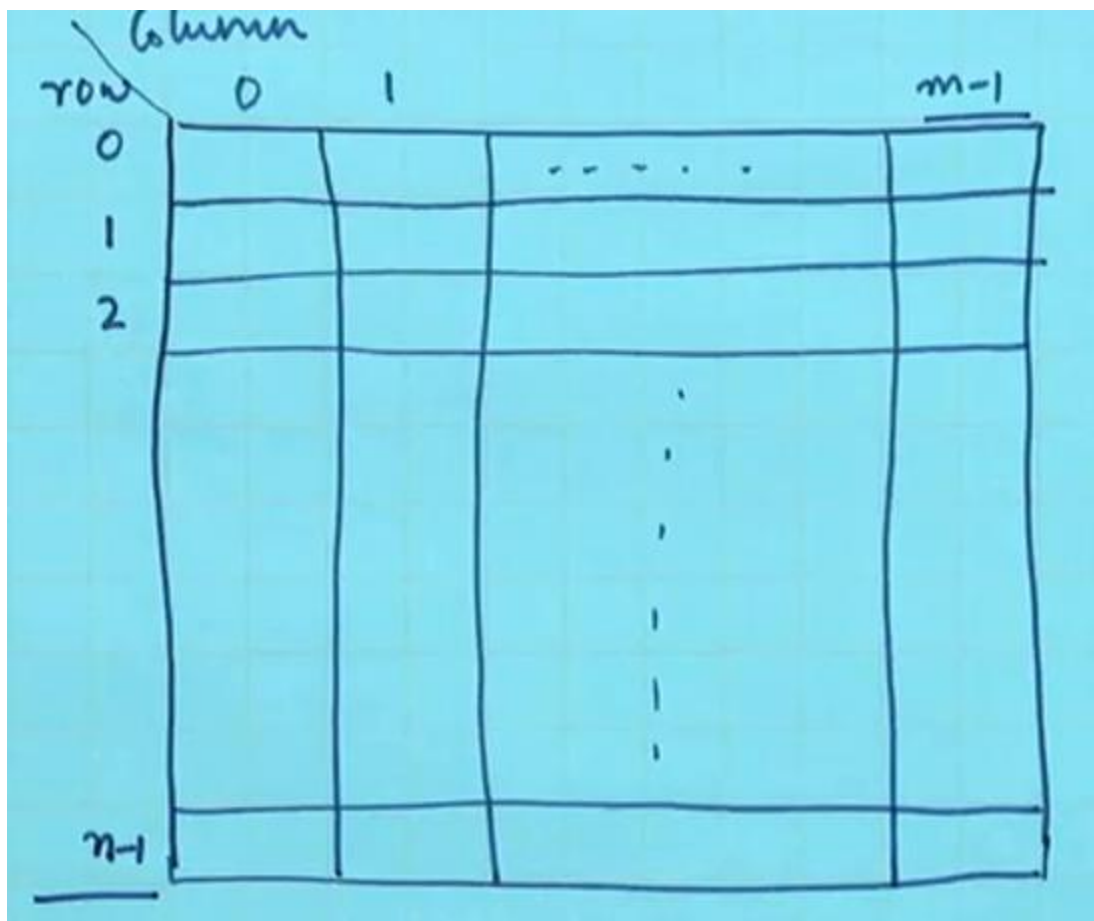
Asynchronous DRAM

- The timing of the memory device is controlled asynchronously.
- The device connected to this memory is responsible for the delay.
- Address lines are divided into two parts and multiplexed.
 - Upper half of address:
 - Loaded into *Row Address Latch* using *Row Address Strobe* (RAS).
 - Lower half of address:
 - Loaded into *Column Address Latch* using *Column Address Strobe* (CAS).

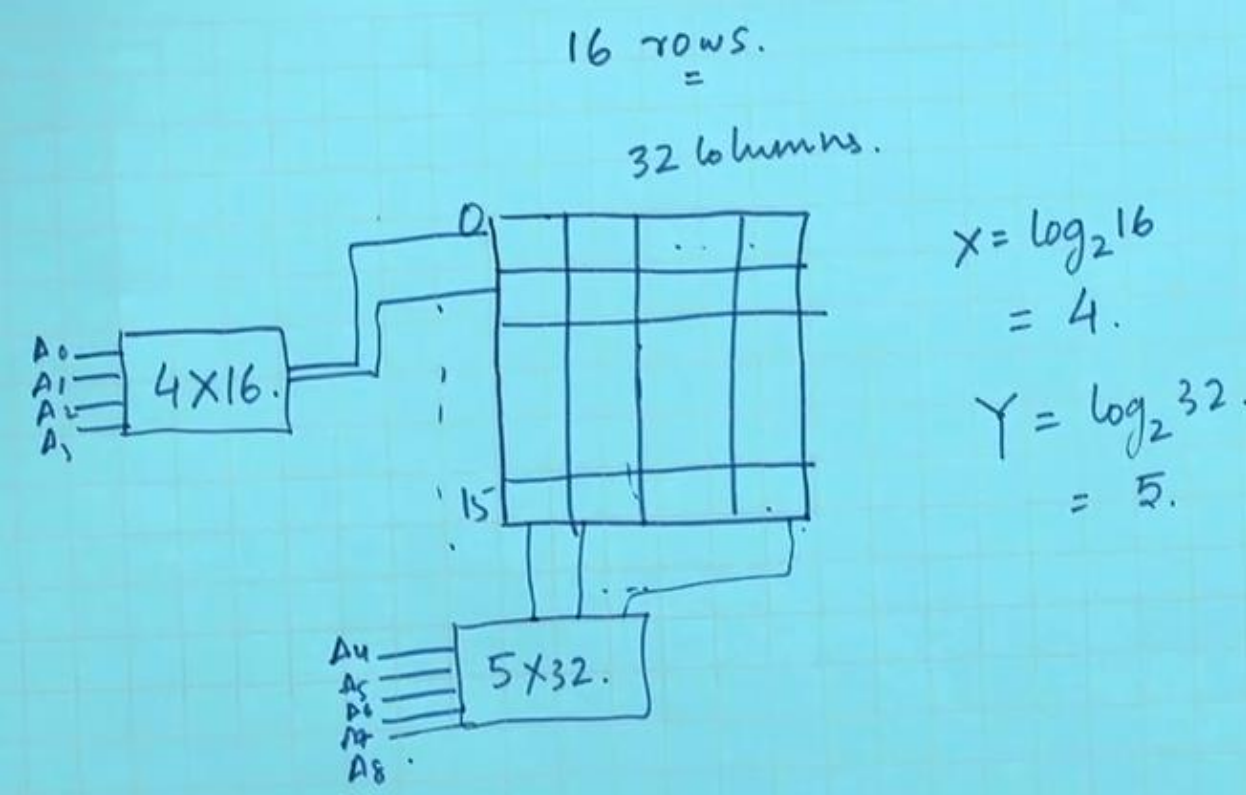
Internal Organization of a DRAM Chip



- Cells are organized in the form of an array, in rows and columns.
- Cells of each row are divided into fixed number of columns, m bits each.
- m is 8, 16, 32 or 64.



- Suppose that the memory cell array is organized as $r \times c$.
— r rows and c columns.
- An x -bit address is required to select a row r , where $x = \log_2 r$.
- An y -bit address is required to select a column c , where $y = \log_2 c$.
- Total address bits: $n = x \text{ (high order)} + y \text{ (low order)}$

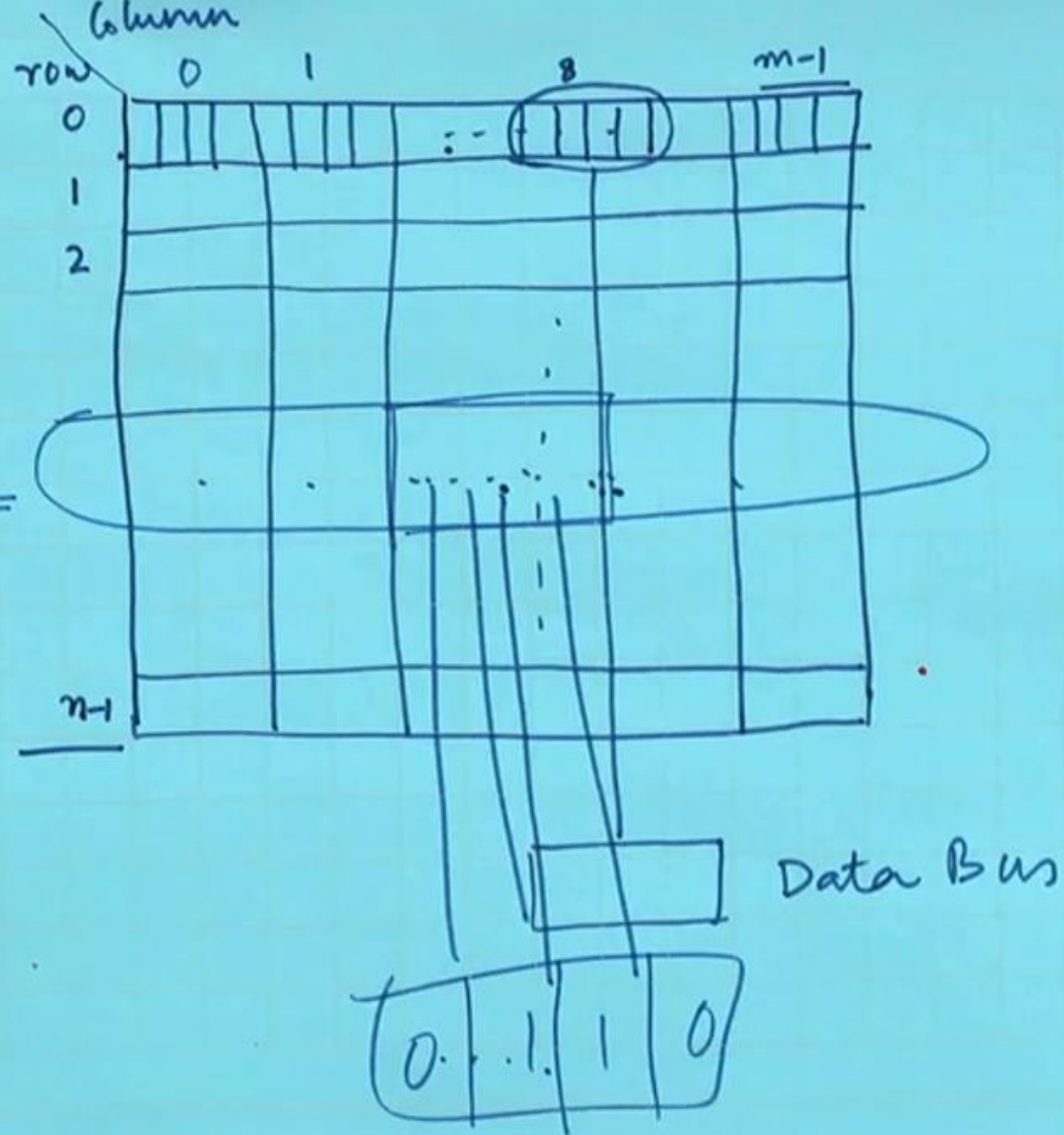


READ or WRITE Operation

- For a read operation, the x -bit row address is applied first.
 - It is loaded into *Row Address Latch* in response to the signal *RAS'*.
 - The read operation is performed in which all the cells of the selected row are read and refreshed.
- After loading of row address, the column address is selected.
- In response to *CAS'* the column address is loaded into *Column Address Latch*.
- Then the column decoder selects a particular column from c columns and an appropriate group of m sense/write circuits are selected.

RAS =

CAS

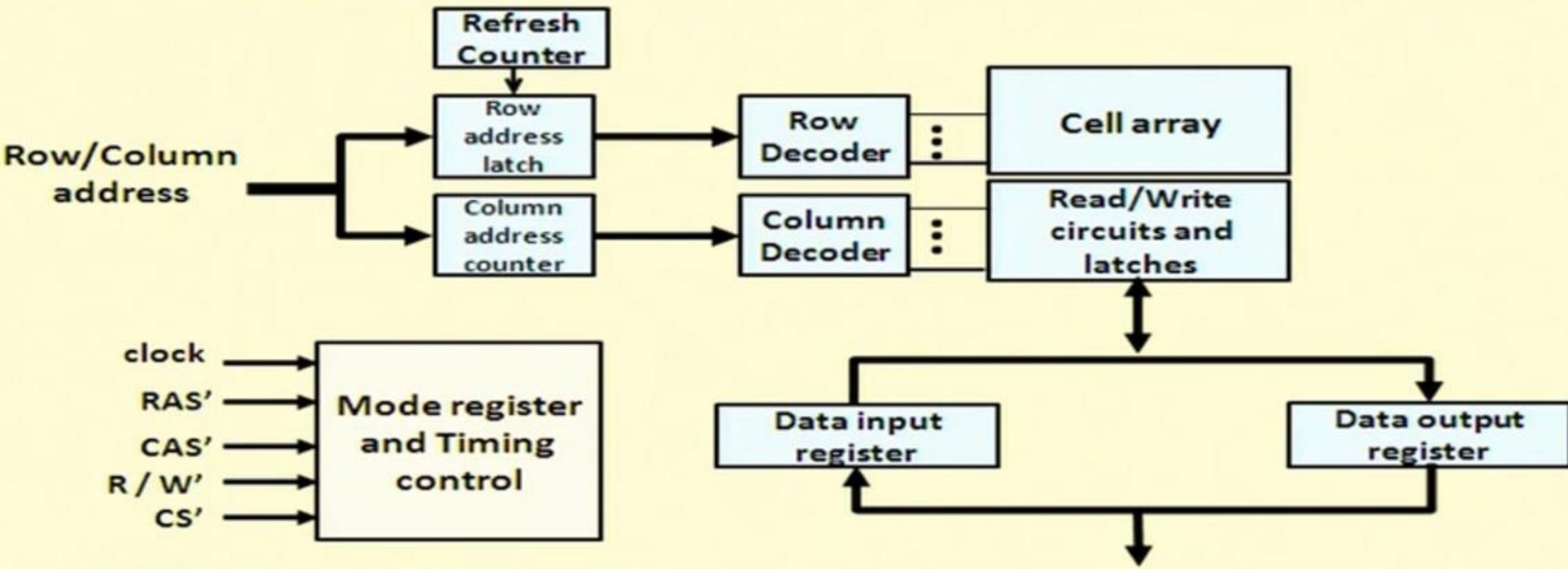


- For a READ operation, the output values of the selected circuits are transferred to data lines D_{m-1} to D_0 .
- For a WRITE operation, the data available on the data lines D_{m-1} to D_0 is transferred to the selected circuits.
 - This information is stored in the selected cell.
- Both RAS' and CAS' are active low signals. That is they cause latching the addresses when they move from high to low.
- Each row of the cell array must be periodically refreshed to prevent data loss.
- Cost is low but access time is high compared to SRAM.
- Very high packing density (few billion cells per chip).
- Widely used in the main memory of modern computer systems.

Synchronous DRAM

- SDRAM is the commonly used name for various kinds of dynamic RAM that are synchronized with clock.
- The structure of this memory is same as asynchronous DRAM.
- The concept of SDRAM were known from 70's but it is first developed by Samsung in the year 1993 (KM48SL2000).
 - By 2000 all kinds of DRAM were replaced by SDRAM.

Internal Organization of a SDRAM Chip



- In SDRAM address and data connections are buffered by registers.
- The output of individual sense amplifier is connected to a latch.
- Mode register is present which can be set to operate the memory chip in different modes.
- To select successive columns it is not required to provide externally generated pulses on CAS line.
- A column counter is used internally to generate the required signals.

READ and WRITE Operations

- For READ operation, the row address is applied first, and in response to the column address, the data present in the latches for the selected columns are transferred to the data output register.
 - Then the data is available on the data bus.
- For WRITE operation, the row address is applied first, and in response to the column address, the data present in the data bus is made available to the latches through data input register.
 - The data is then written to the particular cell.

Read-Only Memories (ROMs)

- SRAM and SDRAM chips are volatile:
 - Lose the contents when the power is turned off.
- Many applications need memory devices to retain contents after the power is turned off.
 - For example, computer is turned on, the operating system must be loaded from the disk into the memory.
 - To load OS into memory, Boot program has to be executed , since Boot pgm is quite large, most of it is stored on the disk. Now processor has to execute some instructions that load boot pgm into memory.
 - Need to store these instructions so that they will not be lost after the power is turned off.
 - We need to store these instructions into a non-volatile memory.
- Non-volatile memory is read in the same manner as volatile memory.
 - But Separate writing process is needed to place information in this memory.
 - Since its Normal operation involves only reading of data, this type of memory is called Read-Only memory (ROM).

ROM: Read Only Memory

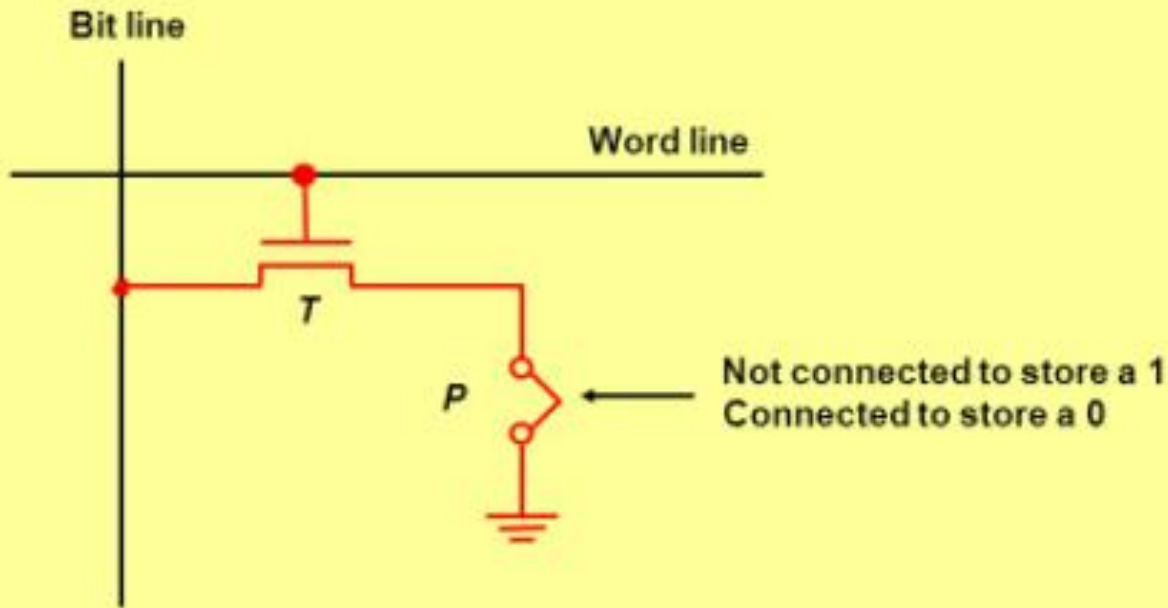


Figure 5.12. A ROM cell.

- A logic Value 0 is stored in the cell if the transistor is connected to ground at point P; otherwise , a 1 is stored (as bit line is connected through a resistor to the power supply.)
- To read the state of the cell, the word line is activated. The transistor switch is closed and the voltage on the bit line drops to near zero if there is a connection between the transistor and ground.
- If there is no connection to ground, the bit line remains at the high voltage , indicating 1 .
- A sense ckt at the end of the bit line generates the proper output value.
- Data are written into a ROM when it is manufacture .

TYPES OF ROM

- Different types of non-volatile memory are
 - 1) PROM
 - 2) EPROM
 - 3) EEPROM &
 - 4) Flash Memory (Flash Cards & Flash Drives)

Read-Only Memories (Contd.,)

PROM (PROGRAMMABLE ROM)

- PROM allows the data to be loaded by the user.
- Programmability is achieved by inserting a „fuse“ at point P in a ROM cell.
- Before PROM is programmed, the memory contains all 0"s.
- User can insert 1"s at required location by burning-out fuse using high current-pulse.
- This process is irreversible.
- **Advantages:**
 - 1) It provides flexibility.
 - 2) It is faster.
 - 3) It is less expensive because they can be programmed directly by the user.

Read-Only Memories (Contd.,)

EPROM (ERASABLE REPROGRAMMABLE ROM)

- EPROM allows
 - stored data to be erased and
 - new data to be loaded. such an erasable, pgmable ROM is usually called an EPROM
- In cell, a connection to ground is always made at „P“ and a special transistor is used.
- The transistor has the ability to function as
 - a normal transistor or
 - a disabled transistor that is always turned „off“.
- Transistor can be programmed to behave as a permanently open switch, by injecting charge into it.
- Erasure requires dissipating the charges trapped in the transistor of memory-cells. that becomes trapped inside
This can be done by exposing the chip to ultra-violet light.
- **Advantages:**
 - 1) It provides flexibility during the development-phase of digital-system.
 - 2) It is capable of retaining the stored information for a long time.
- **Disadvantages:**
 - 1) The chip must be physically removed from the circuit for reprogramming.
 - 2) The entire contents need to be erased by UV light.

Read-Only Memories (Contd.,)

■ Electrically Erasable Programmable Read-Only Memory (EEPROM):

- To erase the contents of EPROMs, they have to be exposed to ultraviolet light.
- Disadvantage with EPROM , Physically removed from the circuit.

EEPROM (ELECTRICALLY ERASABLE ROM)

• **Advantages:**

- 1) It can be both programmed and erased electrically.
- 2) It allows the erasing of all cell contents selectively.

- **Disadvantage:** It requires different voltage for erasing, writing and reading the stored data.

FLASH MEMORY : Applications include hand held computers, Cell phones, Digital Cameras and MP3 players

- In EEPROM, it is possible to read & write the contents of a single cell. **Block of Cells**
- In Flash device, it is possible to read contents of a single cell & write entire contents of a block.
- Prior to writing, the previous contents of the block are erased.

Eg. In MP3 player, the flash memory stores the data that represents sound.

- Single flash chips cannot provide sufficient storage capacity for embedded-system.

• **Advantages:**

- 1) Flash drives have greater density which leads to higher capacity & low cost per bit.
- 2) It requires single power supply voltage & consumes less power.

3)Power consumption of flash memory is very low, making it attractive for use in equipment that is battery-driven.

Read-Only Memories (Contd.,)

- Single flash chips cannot provide sufficient storage capacity for embedded-system.
- **Larger Memory modules consisting of a number of chips are needed.**
- There are 2 methods for implementing larger memory: 1) Flash Cards & 2) Flash Drives
 - 1) Flash Cards**
 - One way of constructing larger module is to mount flash-chips on a small card.
 - Such flash-card have standard interface.
 - The card is simply plugged into a conveniently accessible slot.
 - Memory-size of the card can be 8, 32 or 64MB.
 - Eg: A minute of music can be stored in 1MB of memory. Hence 64MB flash cards can store an hour of music.
 - 2) Flash Drives**
 - Larger flash memory can be developed by replacing the hard disk-drive.
 - The flash drives are designed to fully emulate the hard disk.
 - The flash drives are solid state electronic devices that have no movable parts.

Read-Only Memories (Contd.,)

Advantages:

- 1) They have shorter seek & access time which results in faster response.
- 2) They have low power consumption. ∴ they are attractive for battery driven application.
- 3) They are insensitive to vibration.

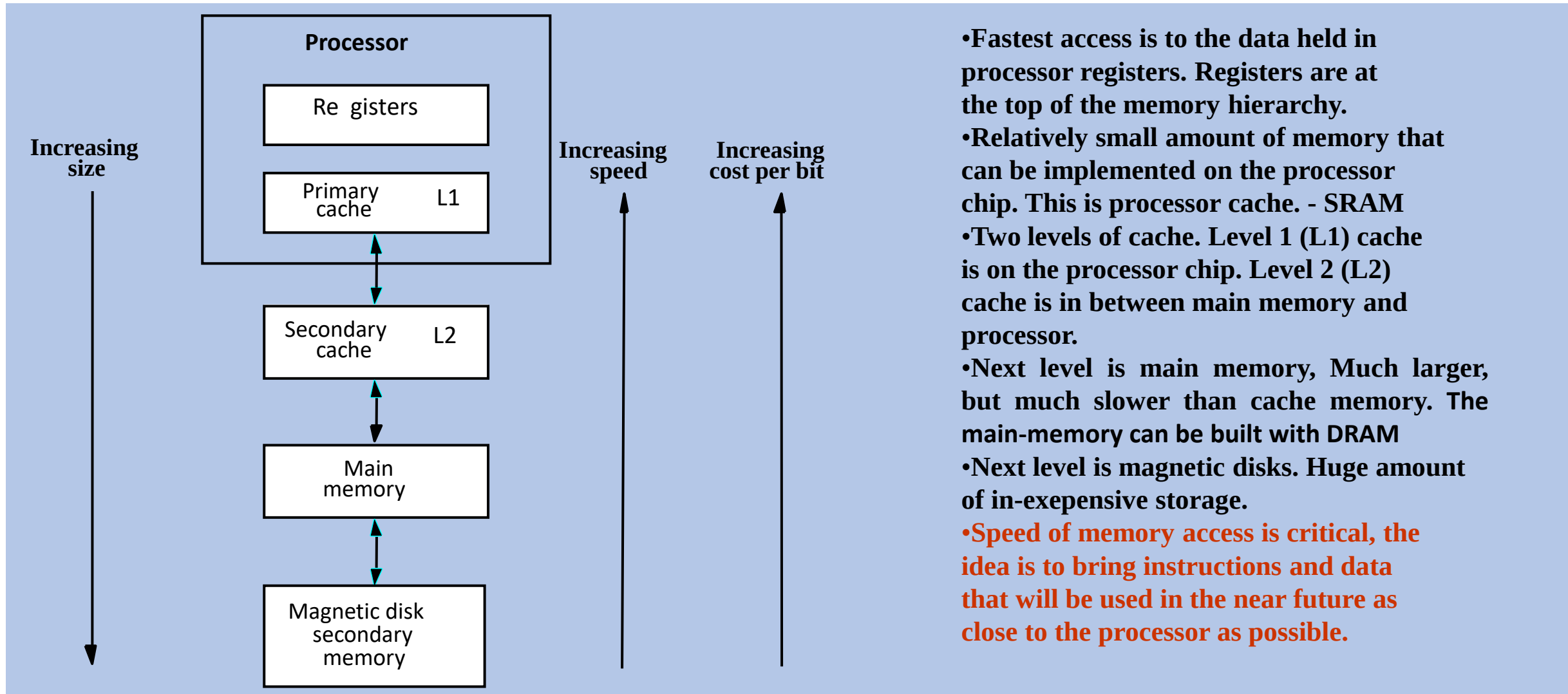
Disadvantages:

- 1) The capacity of flash drive (<1GB) is less than hard disk (>1GB).
- 2) It leads to higher cost per bit.
- 3) Flash memory will weaken after it has been written a number of times (typically at least 1 million times).

Speed, Size, and Cost

- A big challenge in the design of a computer system is to provide a sufficiently large memory, with a reasonable speed at an affordable cost.
- Static RAM:
 - Very fast, but expensive, because a basic SRAM cell has a complex circuit making it impossible to pack a large number of cells onto a single chip.
- Dynamic RAM:
 - Simpler basic cell circuit, hence are much less expensive, but significantly slower than SRAMs.
- Magnetic disks:
 - Storage provided by DRAMs is higher than SRAMs, but is still less than what is necessary.
 - Secondary storage such as magnetic disks provide a large amount of storage, but is much slower than DRAMs.

Memory Hierarchy



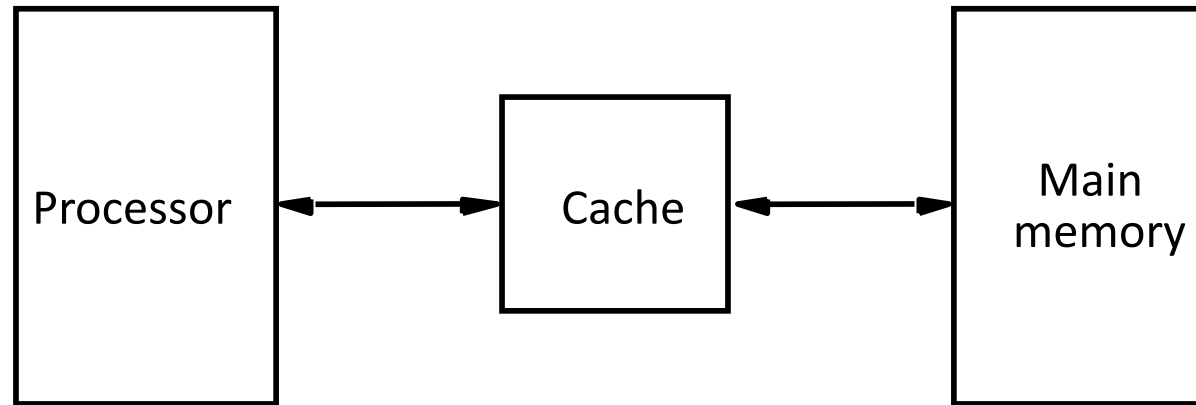
Cache Memories

- Processor is much faster than the main memory.
 - As a result, the processor has to spend much of its time waiting while instructions and data are being fetched from the main memory. – Idling of Processor
 - Major obstacle towards achieving good performance.
- Speed of the main memory cannot be increased beyond a certain point.
- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.
- Cache memory is based on the property of computer programs known as “locality of reference”.

Locality of Reference

- Analysis of programs indicates that many instructions in localized areas of a program are executed repeatedly during some period of time, while the others are accessed relatively less frequently.
 - These instructions may be the ones in a loop, nested loop or few procedures calling each other repeatedly.
 - This is called “locality of reference”.
- **Temporal locality of reference:**
 - Recently executed instruction is likely to be executed again very soon. – whenever an information is required by processor, this info should be brought into the cache where it will remain until it is needed again.
- **Spatial locality of reference:**
 - Instructions with addresses close to a recently executed instruction are likely to be executed soon. – suggest that instead of fetching just one item from the main memory to cache, it is useful to fetch several items that reside at adjacent address as well.

Cache memories



- When Processor issues a Read request, a block of words is transferred from the main memory to the cache, one word at a time.
- Subsequently when the program references any of the locations in this block, the desired contents are read directly from the cache.
- At any given time, only some blocks in the main memory are held in the cache. Which blocks in the main memory are in the cache is determined by a “mapping function”.
- When the cache is full, and a block of words needs to be transferred from the main memory, some block of words in the cache must be replaced. This is determined by a “replacement algorithm”.

Cache hit

- Existence of a cache is transparent to the processor. The processor issues Read and Write requests in the same manner. (In a Read operation, **the main memory is not involved**)
- If the requested data is present in the cache then it is called a **Read or Write hit**.
- Read hit:
 - The data is obtained from the cache.
- Write hit:
 - **Cache has a replica of the contents of the main memory.**
 - **Contents of the cache and the main memory** may be updated simultaneously. This is called **write-through** protocol.
 - Update the contents of the cache, and mark it as updated by setting a bit known as the **dirty bit or modified bit**. The contents of the main memory are updated when this block is replaced from cache. This is **write-back or copy-back** protocol.

Cache miss

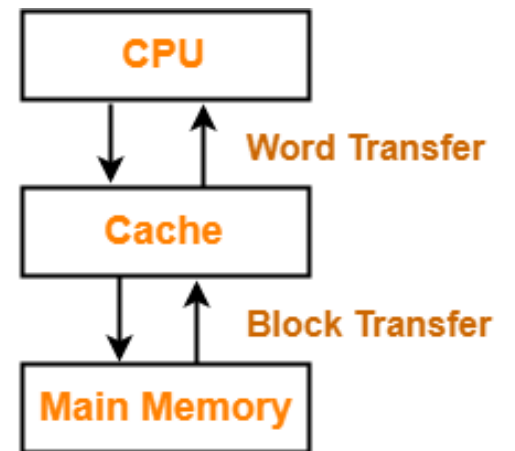
- When the addressed word in the read operation is not in the cache, then a Read miss or Write miss occurs.
- Read miss:
 - Block of words containing this requested word is transferred from the main memory into the cache.
 - After the entire block is transferred into cache, the requested word is forwarded to the processor.
 - The desired word may also be forwarded to the processor as soon as it is read from the main memory, this transfer reduces the processor waiting, this approach is called load-through or early-restart.
- Write-miss:
 - Write-through protocol is used, then the contents of the main memory are updated directly.
 - If write-back protocol is used, the block containing the addressed word is first brought into the cache. The desired word is overwritten with new information.

Cache Memory-

- Cache memory is a Random Access Memory.
- The main advantage of cache memory is its very fast speed.
- It can be accessed by the CPU at much faster speed than main memory.

Location-

- Cache memory lies on the path between the CPU and the main memory.
- It facilitates the transfer of data between the processor and the main memory at the speed which matches to the speed of the processor.
- Data is transferred in the form of words between the cache memory and the CPU.
- Data is transferred in the form of blocks or pages between the cache memory and the main memory.



Cache and Main Memory

Purpose-

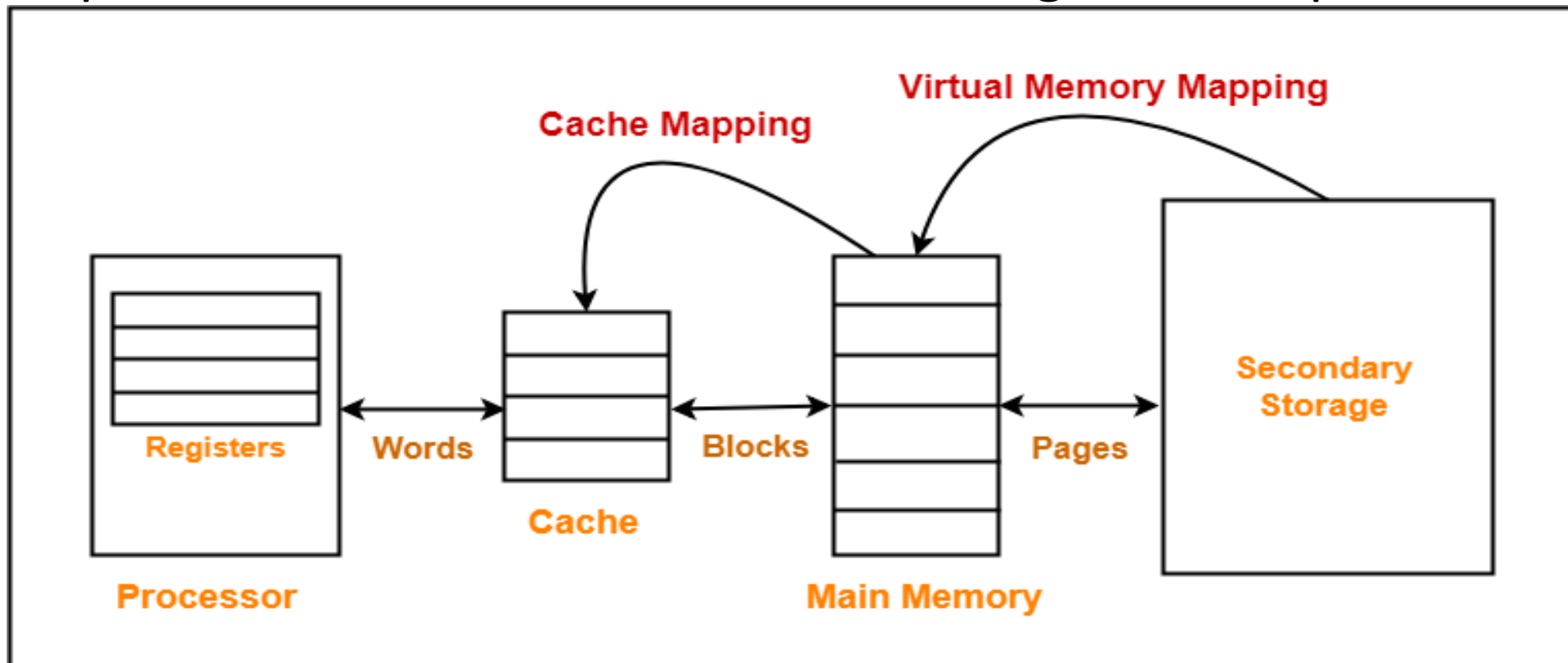
- The fast speed of the cache memory makes it extremely useful.
- It is used for bridging the speed mismatch between the fastest CPU and the main memory.
- It does not let the CPU performance suffer due to the slower speed of the main memory.

Execution Of Program-

- Whenever any program has to be executed, it is first loaded in the main memory.
- The portion of the program that is mostly probably going to be executed in the near future is kept in the cache memory.
- This allows CPU to access the most probable portion at a faster speed.

Step-01:

- Whenever CPU requires any word of memory, it is first searched in the CPU registers.
- If the required word is found in the CPU registers, it is read from there.
- If the required word is not found in the CPU registers, Step-02 is followed.



Step-02:

- When the required word is not found in the CPU registers, it is searched in the cache memory.
- Tag directory of the cache memory is used to search whether the required word is present in the cache memory or not.

Now, there are two cases possible-

Case-01: If the required word is found in the cache memory, the word is delivered to the CPU. This is known as **Cache hit**.

Case-02: If the required word is not found in the cache memory, Step-03 is followed. This is known as **Cache miss**.

Step-03:

- When the required word is not found in the cache memory, it is searched in the main memory.
- Page Table is used to determine whether the required page is present in the main memory or not.
- Now, there are two cases possible

Case-01:

If the page containing the required word is found in the main memory,

- The page is mapped from the main memory to the cache memory.
- This mapping is performed using cache mapping techniques.
- Then, the required word is delivered to the CPU.

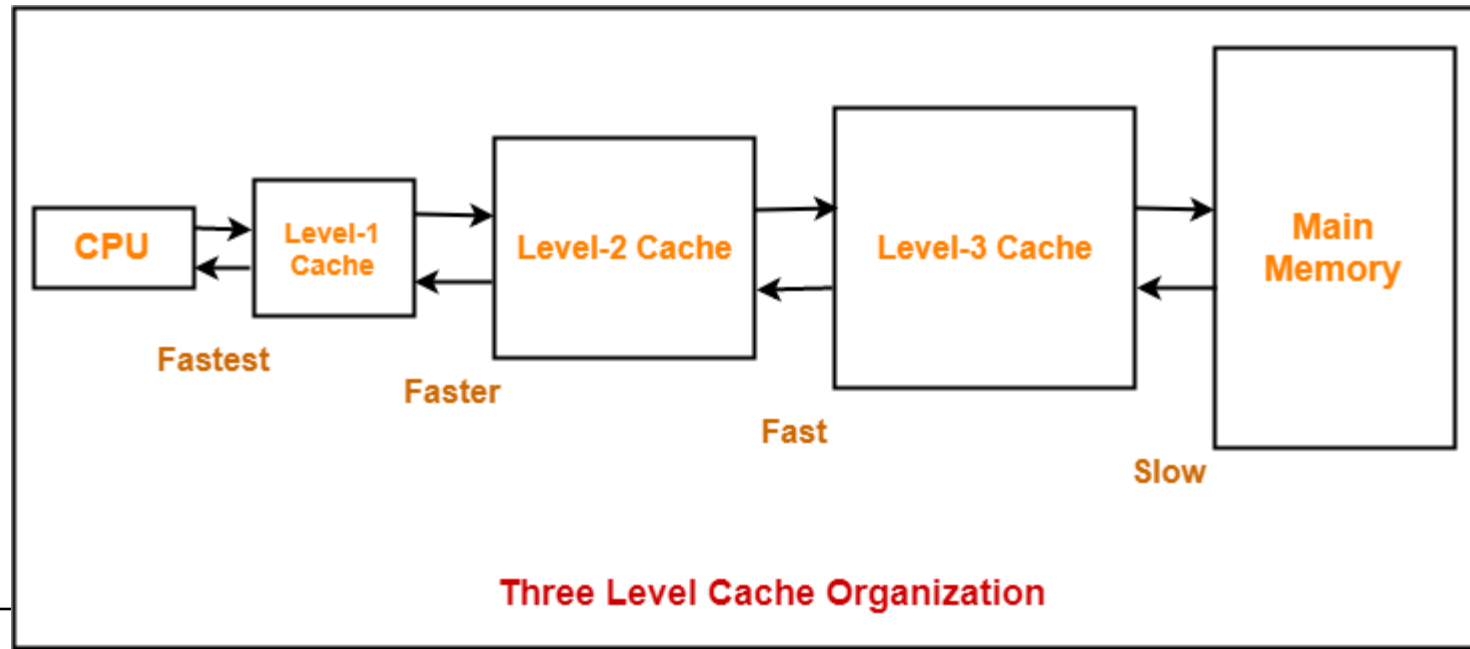
Case-02:

If the page containing the required word is not found in the main memory,

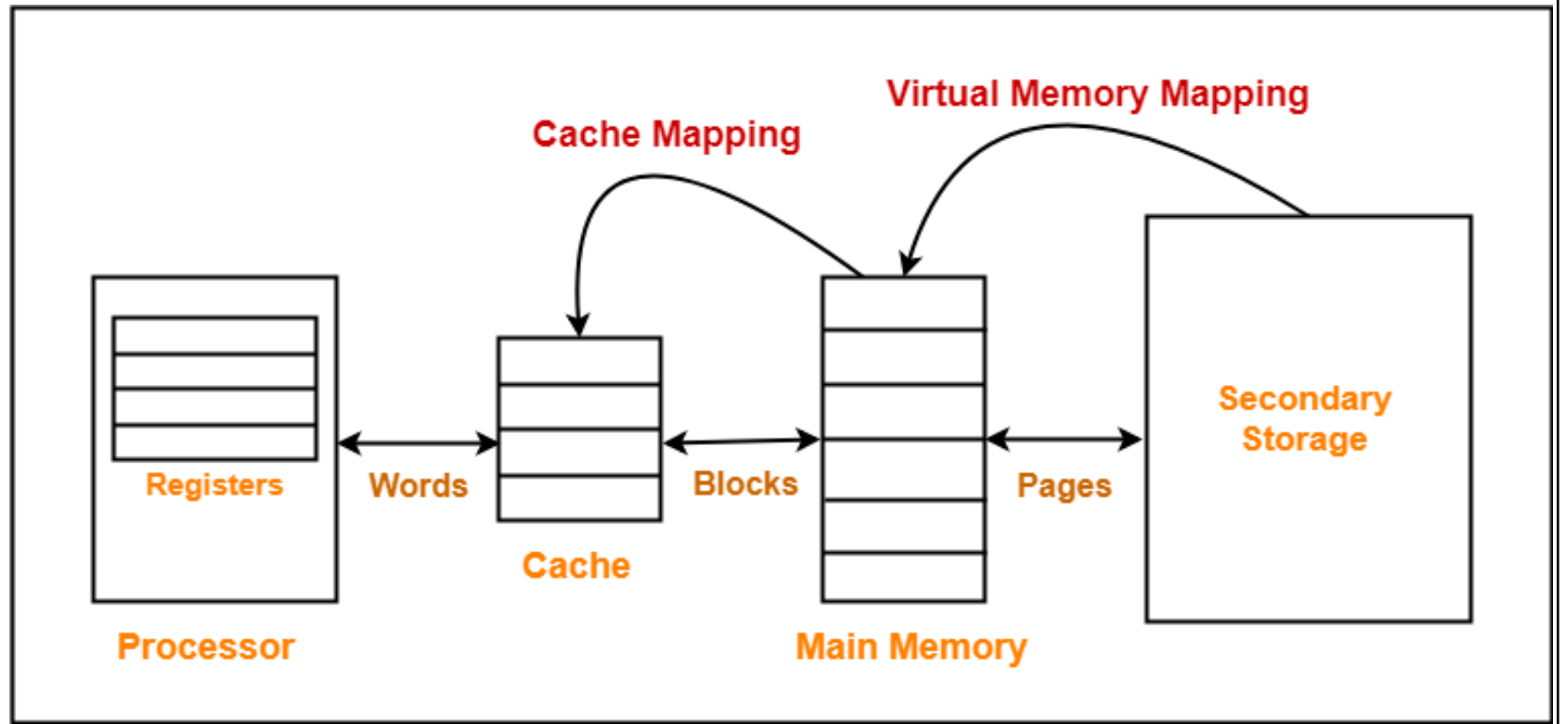
- A page fault occurs.
- The page containing the required word is mapped from the secondary memory to the main memory.
- Then, the page is mapped from the main memory to the cache memory.
- Then, the required word is delivered to the CPU.

Multilevel Cache Organization-

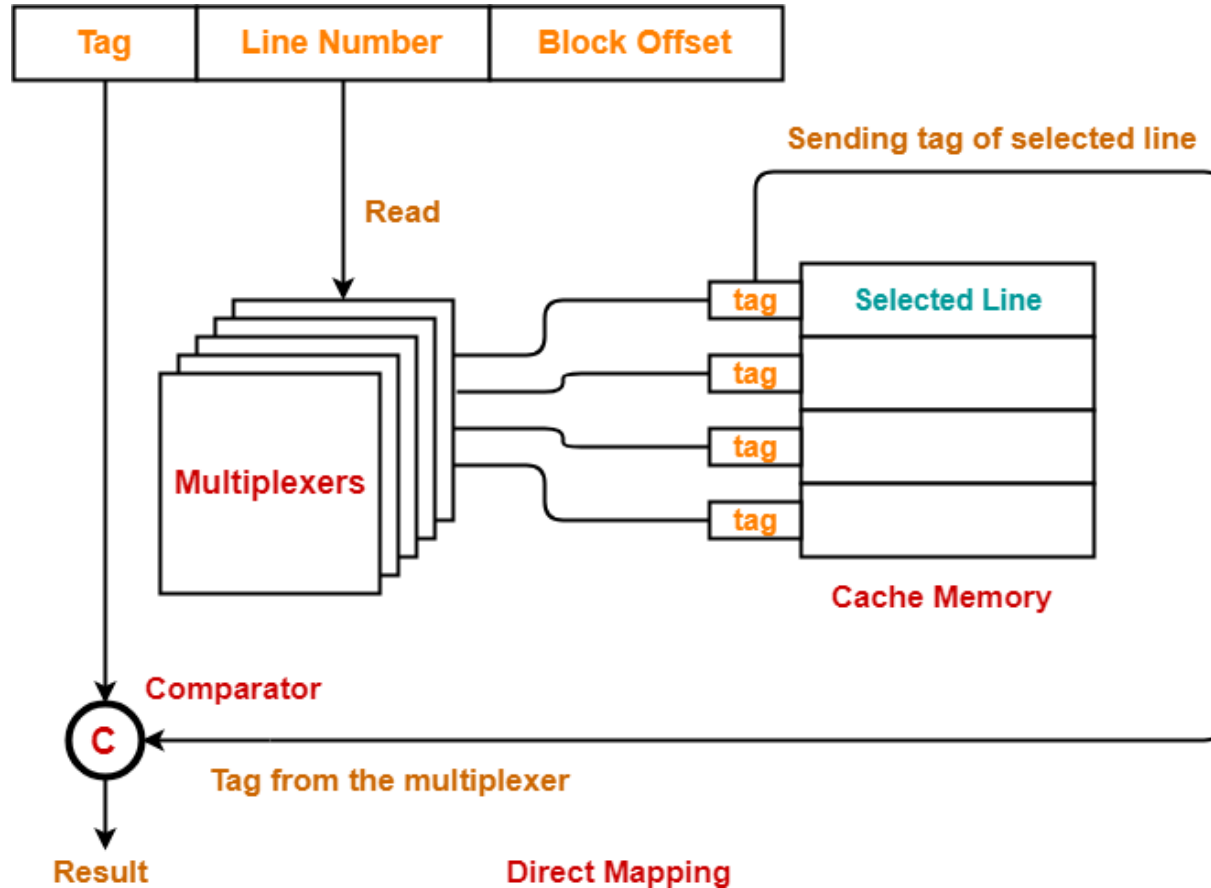
- A multilevel cache organization is an organization where cache memories of different sizes are organized at multiple levels to increase the processing speed to a greater extent.
- The smaller the size of cache, the faster its speed.
- The smallest size cache memory is placed closest to the CPU.
- This helps to achieve better performance in terms of speed.



- Mapping functions determine how memory blocks are placed in the cache.
- Three types of mapping functions:
 - Direct mapping
 - Associative mapping
 - Set-associative mapping.



Mapping Functions : Direct Mapping

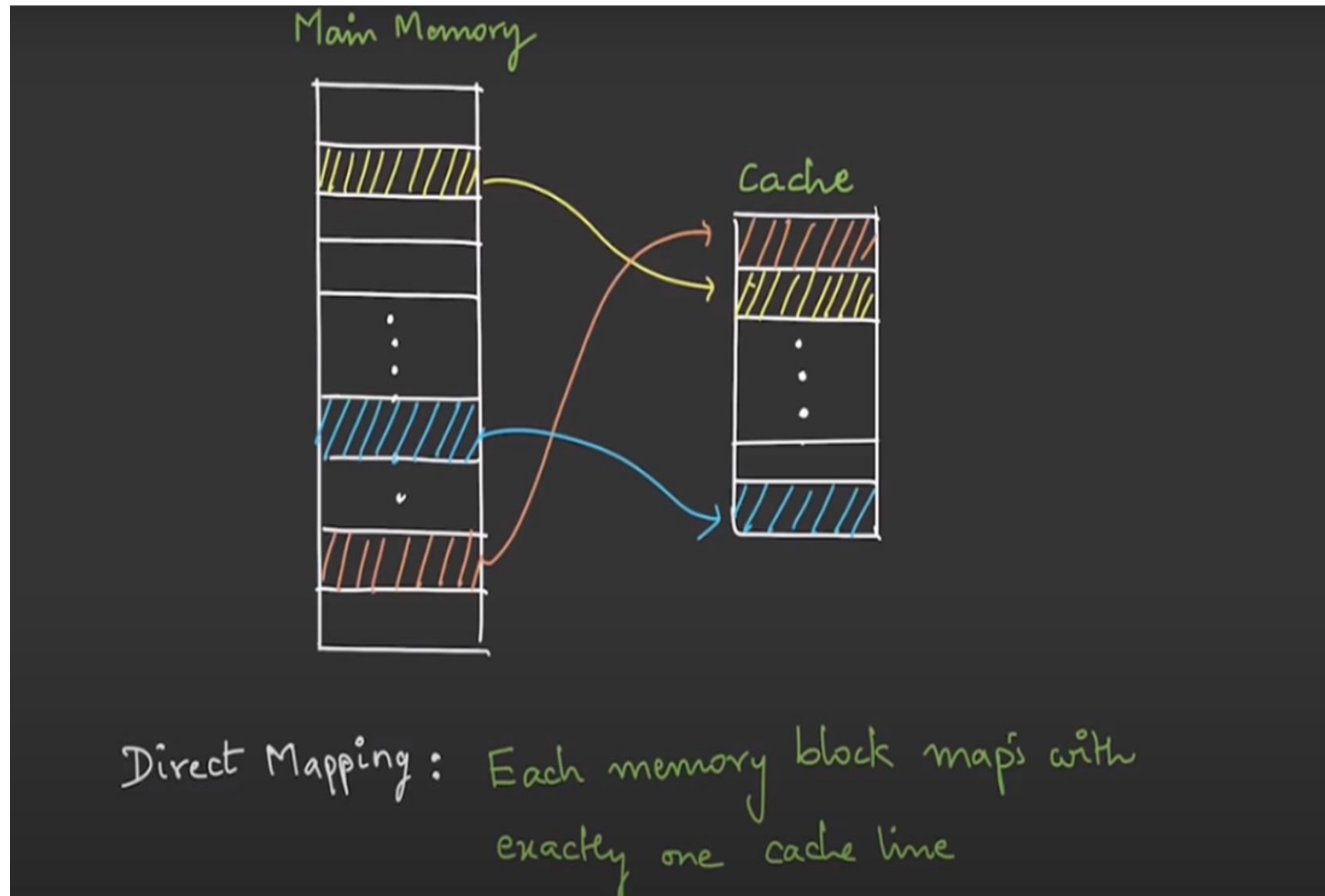


The following steps explain the working of direct mapped cache-

After CPU generates a memory request,

- The line number field of the address is used to access the particular line of the cache.
- The tag field of the CPU address is then compared with the tag of the cache line.
- If the two tags match, a cache hit occurs and the desired word is found in the cache.
- If the two tags do not match, a cache miss occurs.
- In case of a cache miss, the required word has to be brought from the main memory.
- It is then stored in the cache together with the new tag replacing the previous one.

Mapping Functions : Direct Mapping



- Several Words or Bytes are grouped together to form Block (Line in Cache)
- Size of Main Memory Block and Cache Line are same

Example to illustrate direct mapping

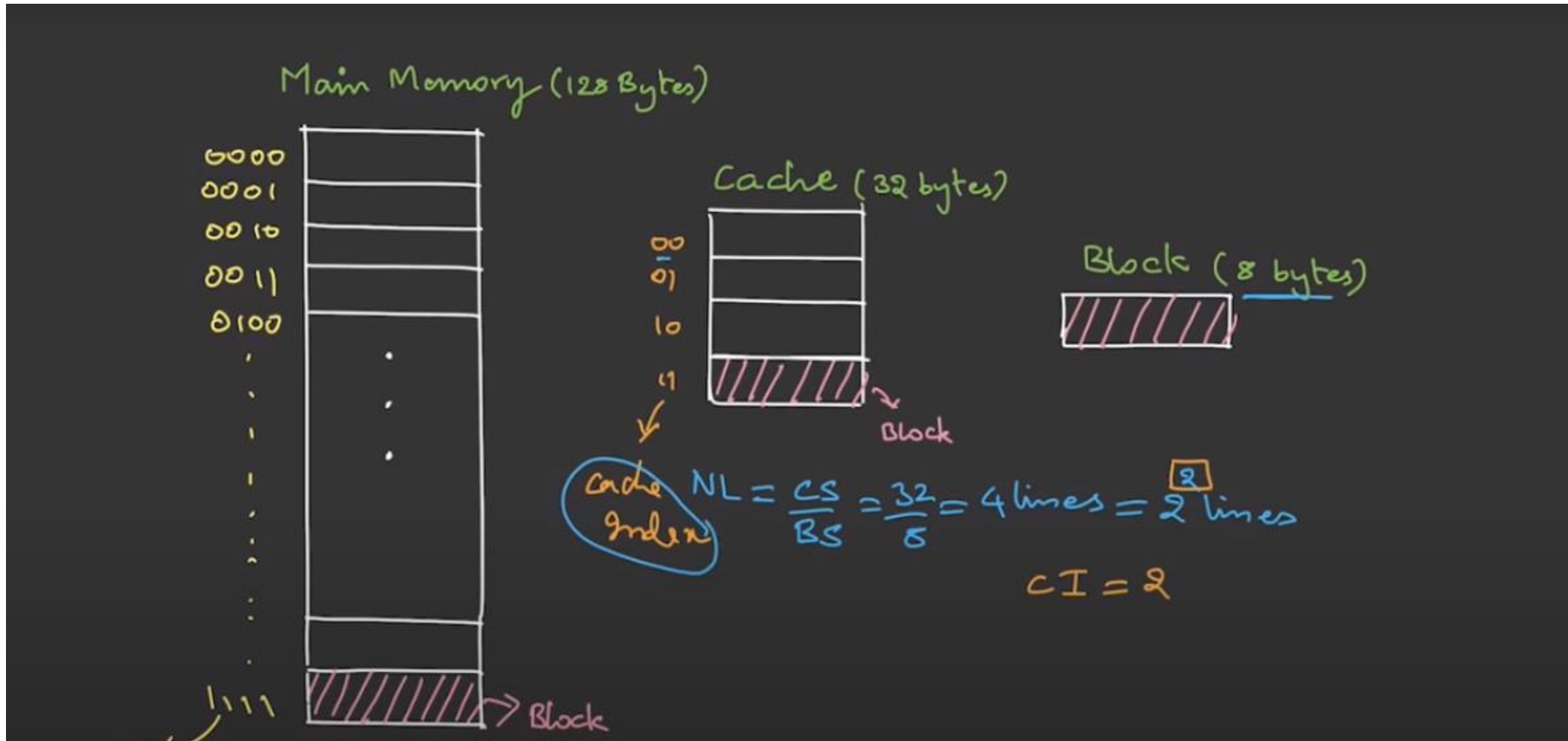
Main Memory (128 Bytes)



Cache (32 bytes)



Block



Number of blocks on main memory = Memory size / Block size

$$= 128 / 8$$

$$= 16 \text{ blocks} = 2^4 \text{ blocks}$$

Therefore Block index bits = 4 bits.

Main memory (128 Bytes)

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Block index (or Block Number)

Cache (32 Bytes)

Cache index (or Line Number)	00	Line-0	Can hold block 0,4,8,12
	01	Line-1	Can hold block 1,5,9,13
	10	Line-2	Can hold block 2,6,10,14
	11	Line-3	Can hold block 3,7,11,15

(0000) Block -0							
8 th Byte	7 th Byte	6 th Byte	5 th Byte	4 th Byte	3 rd Byte	2 nd Byte	1 st Byte
111	110	101	100	011	010	001	000

Since multiple blocks of main memory map to single cache line,CPU needs a mechanism to identify which particular block of main memory is residing in the cache. Hence, it maintains few additional bits called tag bits.

The number of tag bits required is computed using the formula= number of bits in Block index - No of cache index bits

$$= 4-2$$

$$=2$$

Tags hold Most significant bits of block index.

Main memory

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Block index

cache

Cache index {	00	Line-0	→ Can hold block 0,4,8,12
	01	Line-1	→ Can hold block 1,5,9,13
	10	Line-2	→ Can hold block 2,6,10,14
	11	Line-3	→ Can hold block 3,7,11,15

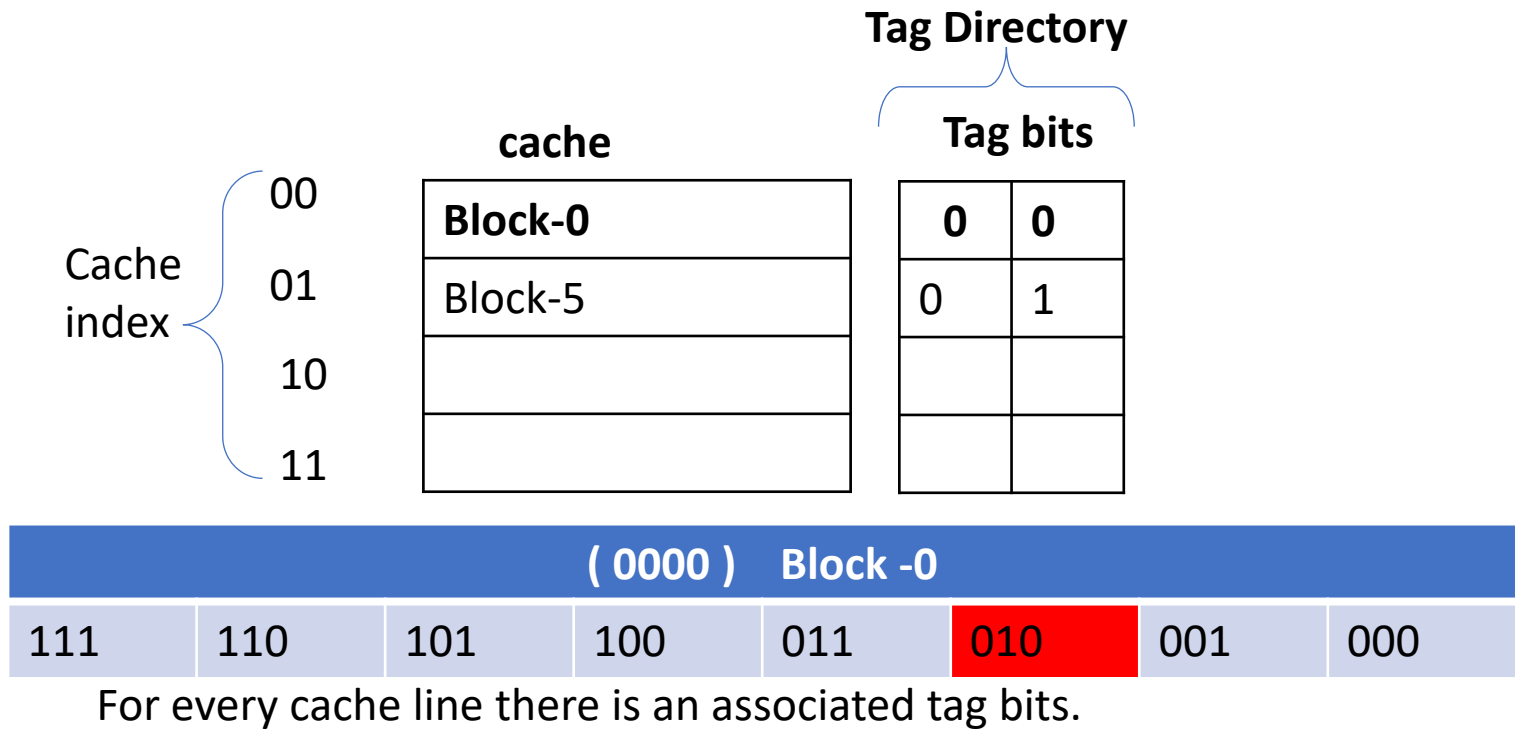
For every cache line there is an associated tag bits.

Tag bits

Suppose, if **Block-1** is placed in line 1 then tag bits will hold 00, similarly for block-4 tag bits are 01 ,for 8 tag bits are 10, and for block-12 tag bits are 11

Tag Bits indicate the processor that which block of main memory is executing in Cache Line.

Block index	0000	Block-0
	0001	Block-1
	0010	Block-2
	0011	Block-3
	0100	Block-4
	0101	Block-5
	0110	Block-6
	0111	Block-7
	1000	Block-8
	1001	Block-9
	1010	Block-10
	1011	Block-11
	1100	Block-12
	1101	Block-13
	1110	Block-14
	1111	Block-15



Let us assume the cache is half filled as shown above and processor generates the request in order

Requests:

1. 0000010
2. 0101011
3. 0100011
4. 0101111

Physical address

Number of bits in physical address is determined by
 Memory size=128 bytes
 $=2^7 = 7$ bits
 Cache size=32 bytes= 2^5
 Cache address= 5 bits.
 Block size=8 bytes
 $=2^3$
 block offset=3 bits.

Main memory (128 Bytes)

Block Index
(Block Nor)

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Tag Directory

Cache (32 Bytes)

Cache index
(Line Nor)

00
01
10
11

Block-0
Block-5

Tag bits

0	0
0	1

For every cache line there is an associated tag bits.

Let us assume the cache is half filled as shown above and processor generates the request in order

Requests:

1. 00 00 010
2. 01 01 011
3. 01 00 011
4. 01 01 111

Physical address

Let us consider the first request

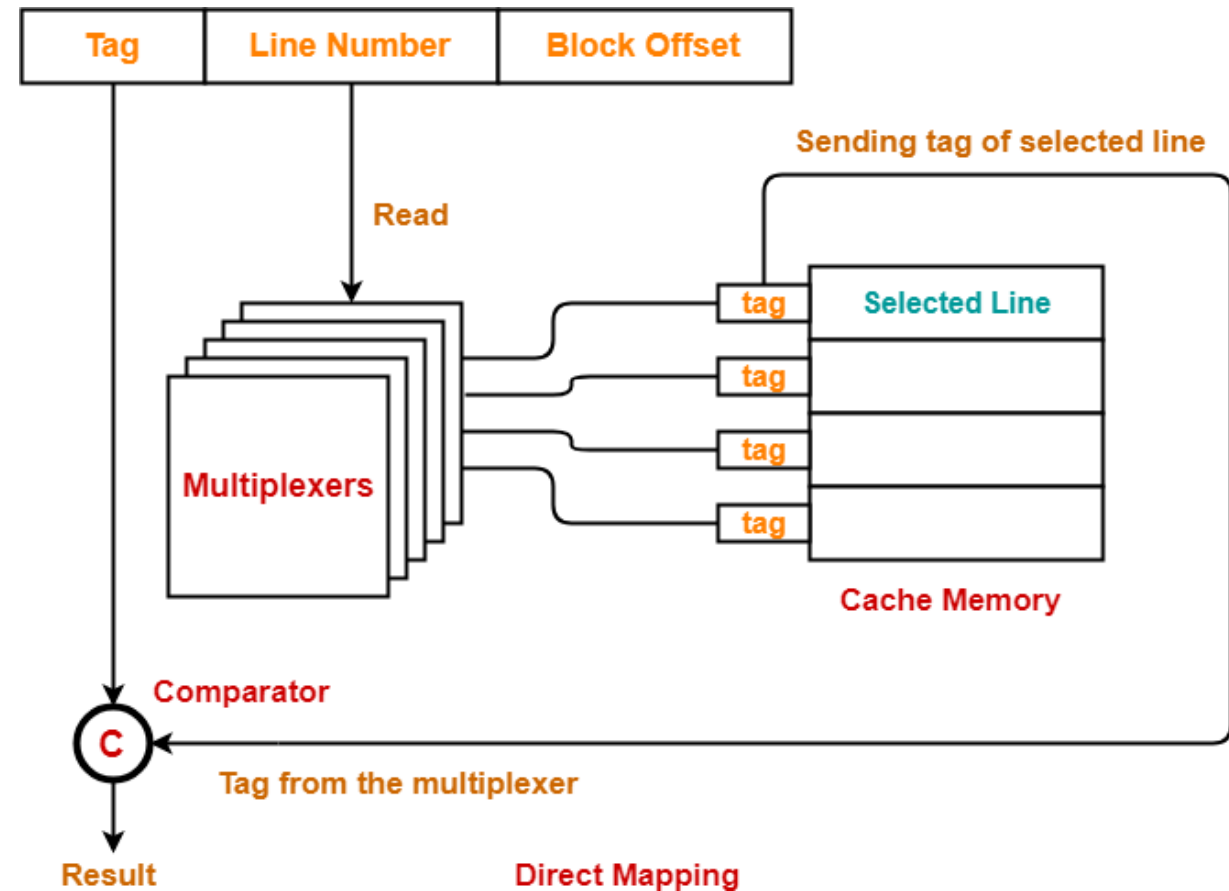
Cache index
(Line Nor)

0	0	0	0	0	1	0
---	---	---	---	---	---	---

Tag bits

Block offset

Mapping Functions : Direct Mapping

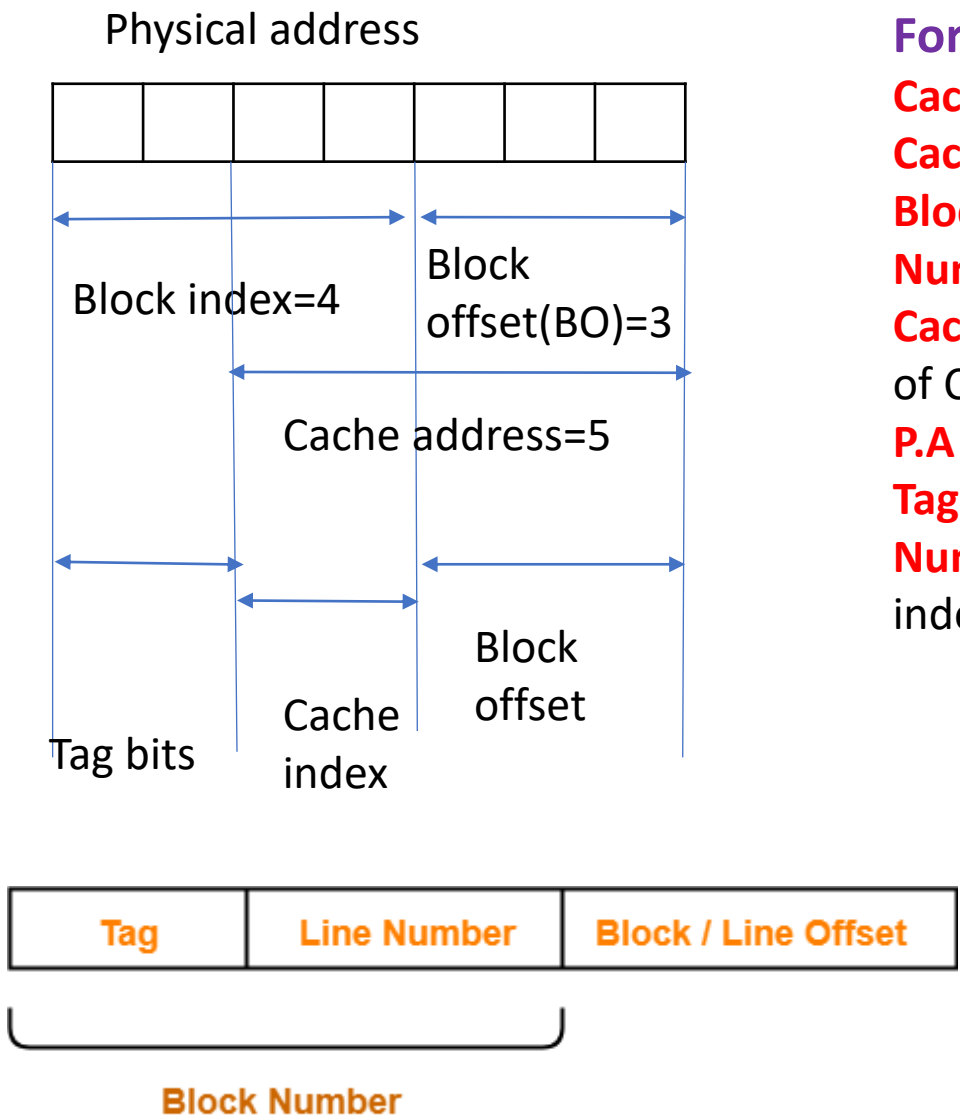


The following steps explain the working of direct mapped cache-

After CPU generates a memory request,

- The **line number field** of the address (PA) is used to access the **particular line of the cache**.
- The **tag field** of the CPU address is then **compared** with the tag of the cache line.
- If the two tags match, a **cache hit occurs** and the desired word is found in the cache.
- If the **two tags do not match**, a **cache miss occurs**.
- In case of a cache miss, the **required word** has to be brought from the **main memory**.
- It is then **stored in the cache** together with the **new tag replacing** the previous one.

Ex : Consider Main Memory of 128 Bytes and Cache Memory of 32 Bytes



Division of Physical Address in Direct Mapping

(0000) Block -0							
111	110	101	100	011	010	001	000

- Formulas :**
- Block offset Bits** = Block Size in terms of 2^N
 - Cache address bits** = Cache Size in terms of 2^N
 - Cache Index** = Cache address bits – Block offset bits
 - Block index bits** = P.A bits – Block offset bits
 - Number of Cache Lines (Blocks)** = $2^{\text{Cache Index bits}}$
 - Cache Size** = $2^{\text{Cache Address bits}}$ or (Block Size in Bytes * Total Number of Cache Lines)
 - P.A bits** = Main memory size in terms of 2^N
 - Tag Directory Size**= Nor of Cache lines * Nor of Tag Bits
 - Number of bits in tag bits** = Physical address - cache address bits or(block index- cache index)

		Tag Directory	
		Tag bits	
Cache index	00	Line-0	0 0
	01	Line-1	0 1
	10	Line-2	
	11	Line-3	

Problems on direct mapping

1.Consider a direct mapped cache of size 2KB with block size 16 bytes. The main memory has 64KB. Find The number of bits needed for i) Cache index ii) Block Offset iii) Cache Address iv) Tag bits v) Block Index and vi) Physical Address

Solution: Given Cache Size 2 KB = $2^1 * 2^{10} = 11$ bits for cache addressing (iii)

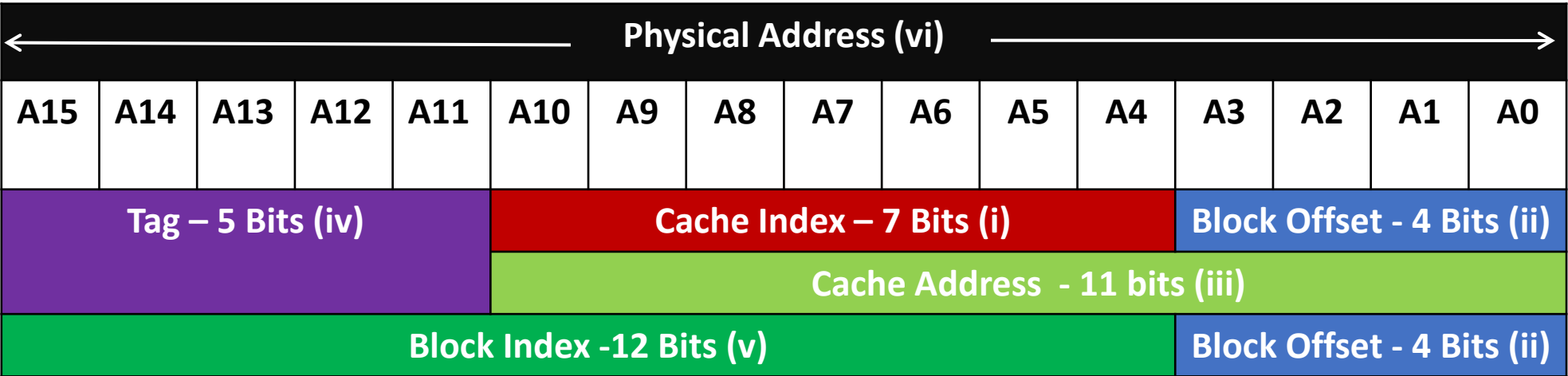
Block Size is 16bytes = $2^4 = 4$ bits for block offset (ii)

Therefore required Cache index bits = $11 - 4 = 7$ bits (i)

Physical address bits = main memory size = 64 KB = $2^6 * 2^{10} = 16$ bits (vi)

Tag Bits = PA – CA = $16 - 11 = 05$ bits (iv)

Block index bits = PA – Block offset = $16 - 4 = 12$ bits (v)



Problems on direct mapping

2. Consider a direct mapped cache of size 32KB with block size 32 bytes. The CPU generates 32-bit address. Find The number of bits needed for cache indexing, and the number of tag bits.

Solution: PA bits are 32 bits – Means Main memory size is $= 2^{32}$ Bytes

Given, cache size = $32\text{KB} = 2^5 * 2^{10} = 2^{15}$ bytes

Therefore Cache address = 15 bits

Block size = 32 bytes = 2^5 bytes.

therefore block offset = 5 bits

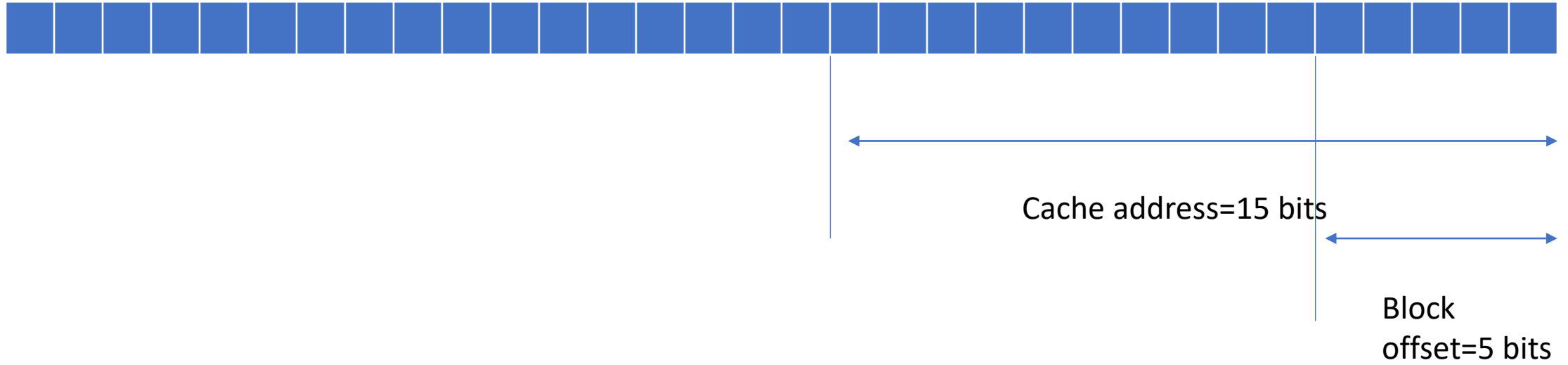
Given physical address = 32 bits.

WKT tag bits = PA - CA

$= 32 - 15$

therefore, number of tag bits = 17 bits.

Physical address=32 bits



Cache index=Cache address – Block offset

= 15-5

Cache index=10 bits

Problem-3: Consider a direct mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-

- Number of bits in tag
- Tag directory size

Solution:

- Cache memory size = 16 KB
- Block size = 256 bytes
- Main memory size = 128 KB
- Size of main memory=128KB
$$=(2^7)*(2^{10}) \text{ bytes}$$
$$=2^{17} \text{ bytes.}$$

Therefore, number of bits in physical address=17 bits.

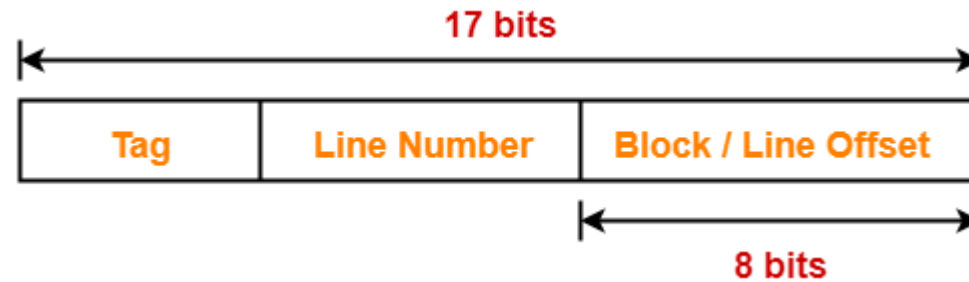
To compute Number of bits in block offset

We have, Block size

= 256 bytes

= 2^8 bytes

Therefore, block offset= 8 bits.



To compute number of bits in cache index

Total number of lines in cache

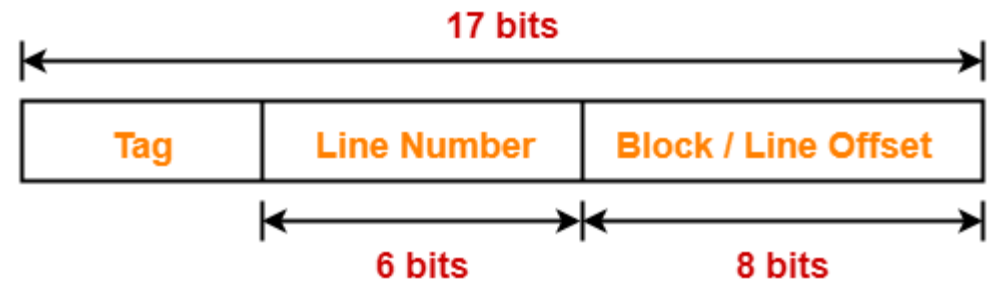
= Cache size / Line size(block size)

= 16 KB / 256 bytes

= 2^{14} bytes / 2^8 bytes

= 2^6 lines

Thus, Number of bits in cache index = 6 bits



To compute Number of Bits in Tag:

Number of bits in tag

= Number of bits in physical address – (Number of bits in cache index + Number of bits in block offset)

= 17 bits – (6 bits + 8 bits)

= 17 bits – 14 bits

= 3 bits

Thus, Number of bits in tag = 3 bits

Or

Tag bits= Physical address- cache address=17-14=3

To compute Tag Directory Size:

Tag directory size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= $2^6 \times 3$ bits

= 192 bits

= 24 bytes

Thus, size of tag directory = 24 bytes

Problem-4:

Consider a direct mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find-

- Size of main memory
- Tag directory size

Solution:

Cache size = 512 KB = $2^9 * 2^{10} = 2^{19}$ Bytes = 19 bits Cache Addr Bits

Block Size = 1 KB = 2^{10} Bytes = 10 bits Block Offset

Number of bits in physical address = $19 + 07 = 26$ bits

Thus, Size of main memory

= 2^{26} bytes

= 64 MB

To compute Tag Directory Size

Tag directory size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

= $2^9 \times 7$ bits

= 3584 bits

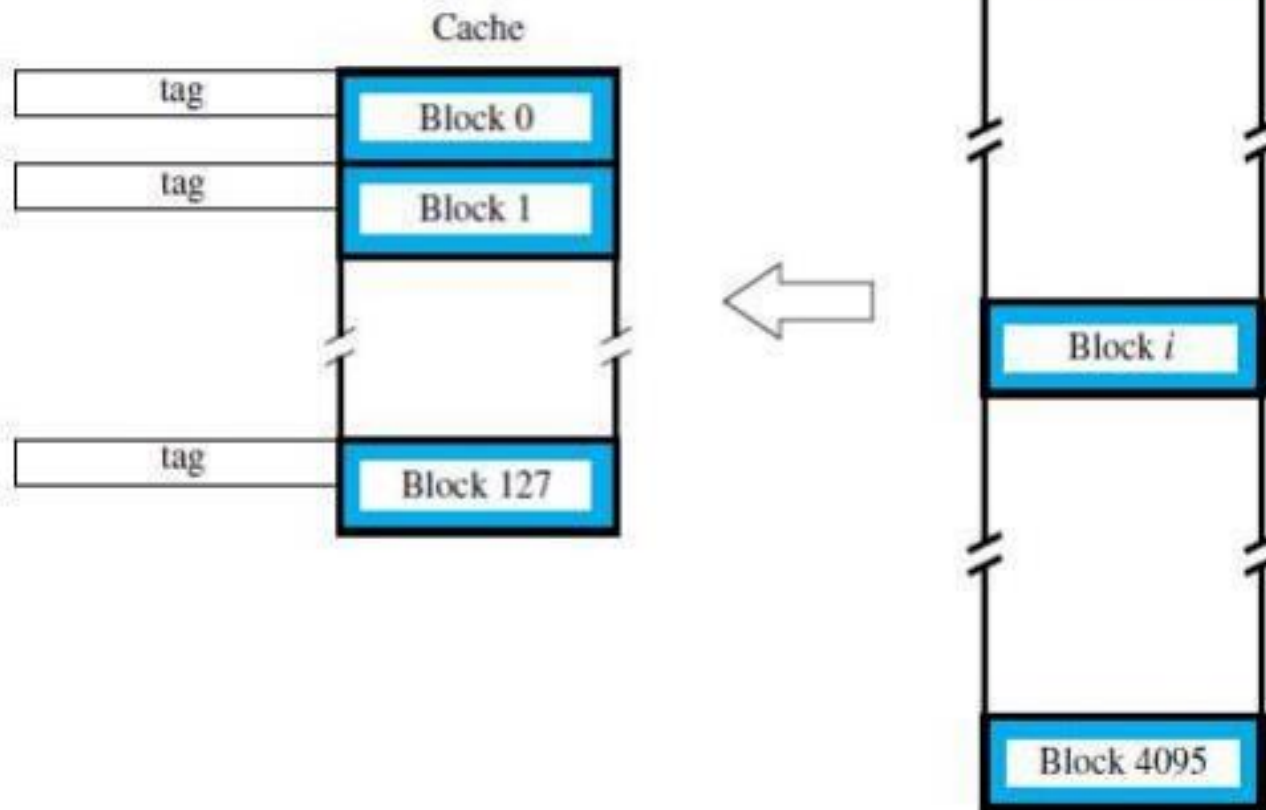
= 448 bytes

Thus, size of tag directory = 448 bytes

Mapping Functions : Associative Mapping

- In associative mapping, A **main memory block** can be placed into any **cache block** position that is freely available at that moment.
- This makes associative mapping **more flexible than direct mapping**. Thus space in cache can be used more efficiently.
- **The tag bits of an address** received from the processor (PA) are compared to the **tag bits of each block of the cache** to see if the desired block is present. This is called **associative mapping technique**.
- A **new block** that has to be brought into the cache has to **replace an existing block only if the cache is full**.
- Thus, replacement algorithm like **FCFS Algorithm, LRU Algorithm** etc is employed.
- The **cost** of an associative cache is **higher than** the cost of a direct mapped cache because of the **need to search all tag patterns** to determine whether a given block is in cache. A search of this kind is called **associative search**.

Consider Cache Consisting of 2 KB with block size 16 Bytes each . Main memory of 64 KB

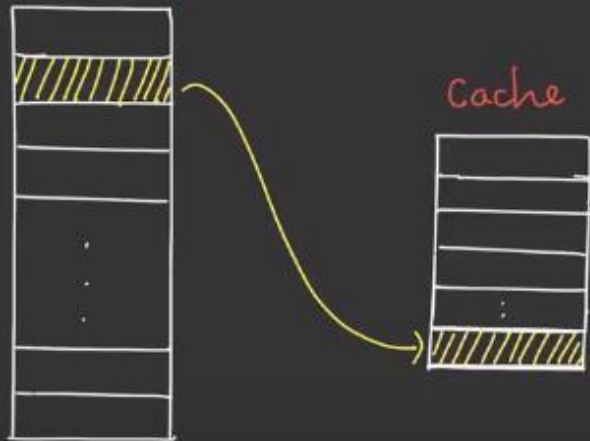


Block Number / Tag	Block / Line Offset
--------------------	---------------------

Division of Physical Address in Fully Associative Mapping

- ◆ **An address is partitioned to**
 - ❖ offset within block
 - ❖ block number
- ◆ **Each block may be mapped to each of the cache lines**
 - ❖ Lookup block in all lines
- ◆ **Each cache line has a tag**
 - ❖ All tags are compared to the block# in parallel
 - ❖ Need a comparator per line
 - ❖ If one of the tags matches the block#, we have a hit
 - Supply data according to offset

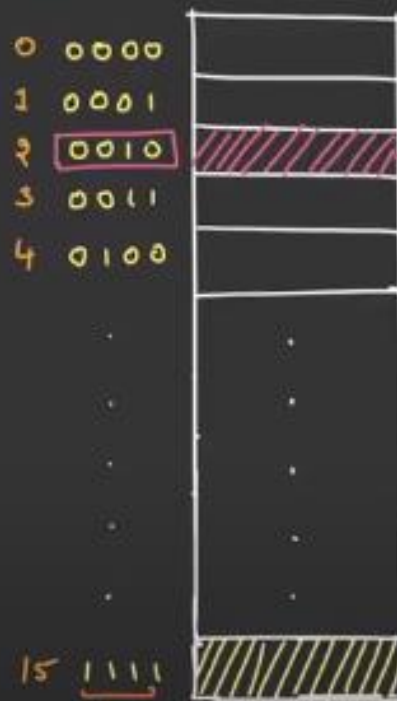
Main Memory



Cache

In associative mapping a memory block can be placed in any cache line

Main Memory (128 Bytes)



15 1111
↓
BI=4

Block
[8 bytes]

Cache (32 bytes)



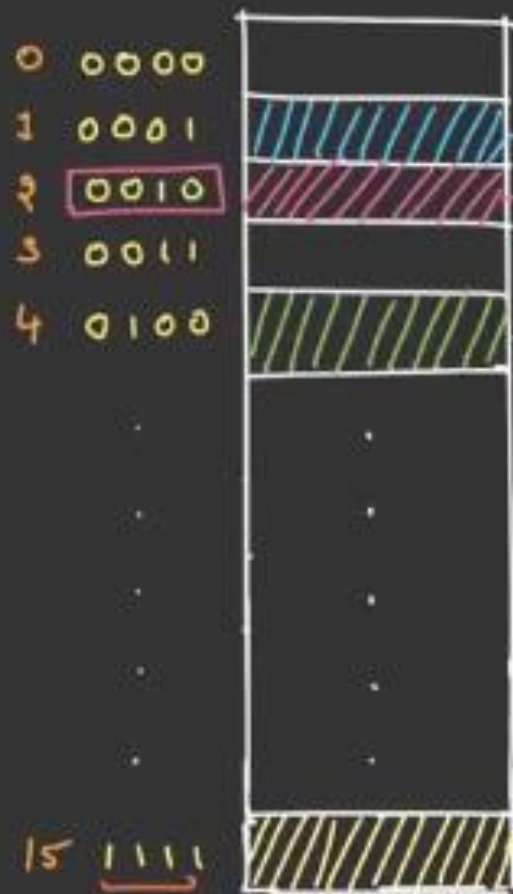
hold block index bits

↑
Tag Bits (4 bits)



TB = BI

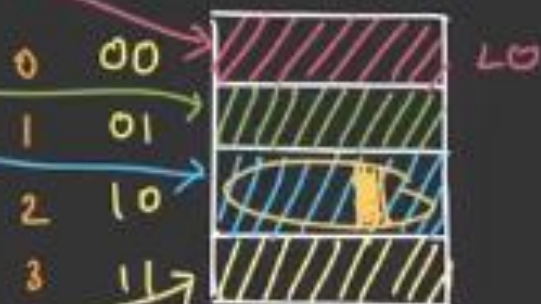
Main Memory (128 Bytes)



$BI = 4$

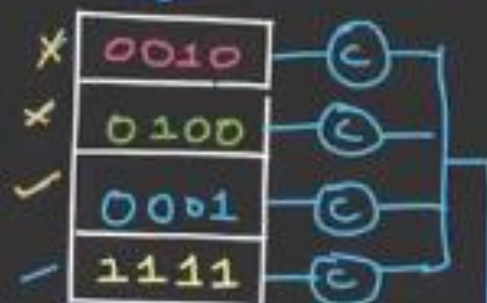
Block
[8 bytes]
 $2^3 \rightarrow B_0$

Cache (32 bytes)



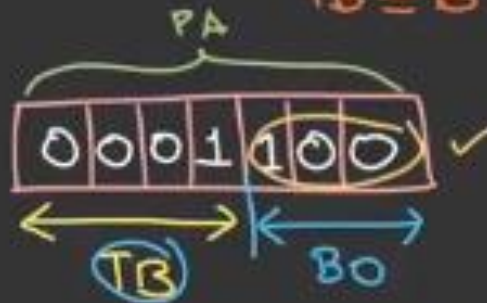
hold block index bits

Tag Bits (4 bits)



$TB = BI$

CPU



Hit

Main memory (128 Bytes)

Block Index
(Block Nor)

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Tag Directory

Cache (32 Bytes)

Cache index
(Line Nor)

00
01
10
11

Block-0
Block-1
Block-2
Block-10

Tag bits

0	0	0	0
0	0	0	1
0	0	1	0
1	0	1	0

For every cache line there is an associated tag bits.

Let us assume the cache is fully filled as shown above and processor generates the request in order

Requests:

1. **0000 010**
2. **0101 011**

Physical address

Let us consider the first request

Physical Address (PA)

0	0	0	0	0	1	0
---	---	---	---	---	---	---

Tag bits

Block offset

Cache Hit

Main memory (128 Bytes)

Block Index
(Block Nor)

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Tag Directory

Cache (32 Bytes)

Cache index
(Line Nor)

00
01
10
11

Block-0
Block-1
Block-2
Block-10

Tag bits

0	0	0	0
0	0	0	1
0	0	1	0
1	0	1	0

For every cache line there is an associated tag bits.

Let us assume the cache is fully filled as shown above and processor generates the request in order

Requests:

1. 0000 010
2. 1010 011

Physical address

Let us consider the first request

Physical Address (PA)

1	0	1	0	0	1	1
---	---	---	---	---	---	---

Tag bits

Block offset

Cache Hit

Main memory (128 Bytes)

Block Index
(Block Nor)

0000	Block-0
0001	Block-1
0010	Block-2
0011	Block-3
0100	Block-4
0101	Block-5
0110	Block-6
0111	Block-7
1000	Block-8
1001	Block-9
1010	Block-10
1011	Block-11
1100	Block-12
1101	Block-13
1110	Block-14
1111	Block-15

Tag Directory

Cache (32 Bytes)

Cache index
(Line Nor)

00
01
10
11

Block-0
Block-1
Block-2
Block-10

Tag bits

0	0	0	0
0	0	0	1
0	0	1	0
1	0	1	0

For every cache line there is an associated tag bits.

Let us assume the cache is fully filled as shown above and processor generates the request in order

Requests:

1. 0000 010
2. 0101 011
3. 0011 001

Physical address

Let us consider the first request

Physical Address (PA)

0	0	1	1	0	0	1
---	---	---	---	---	---	---

Tag bits

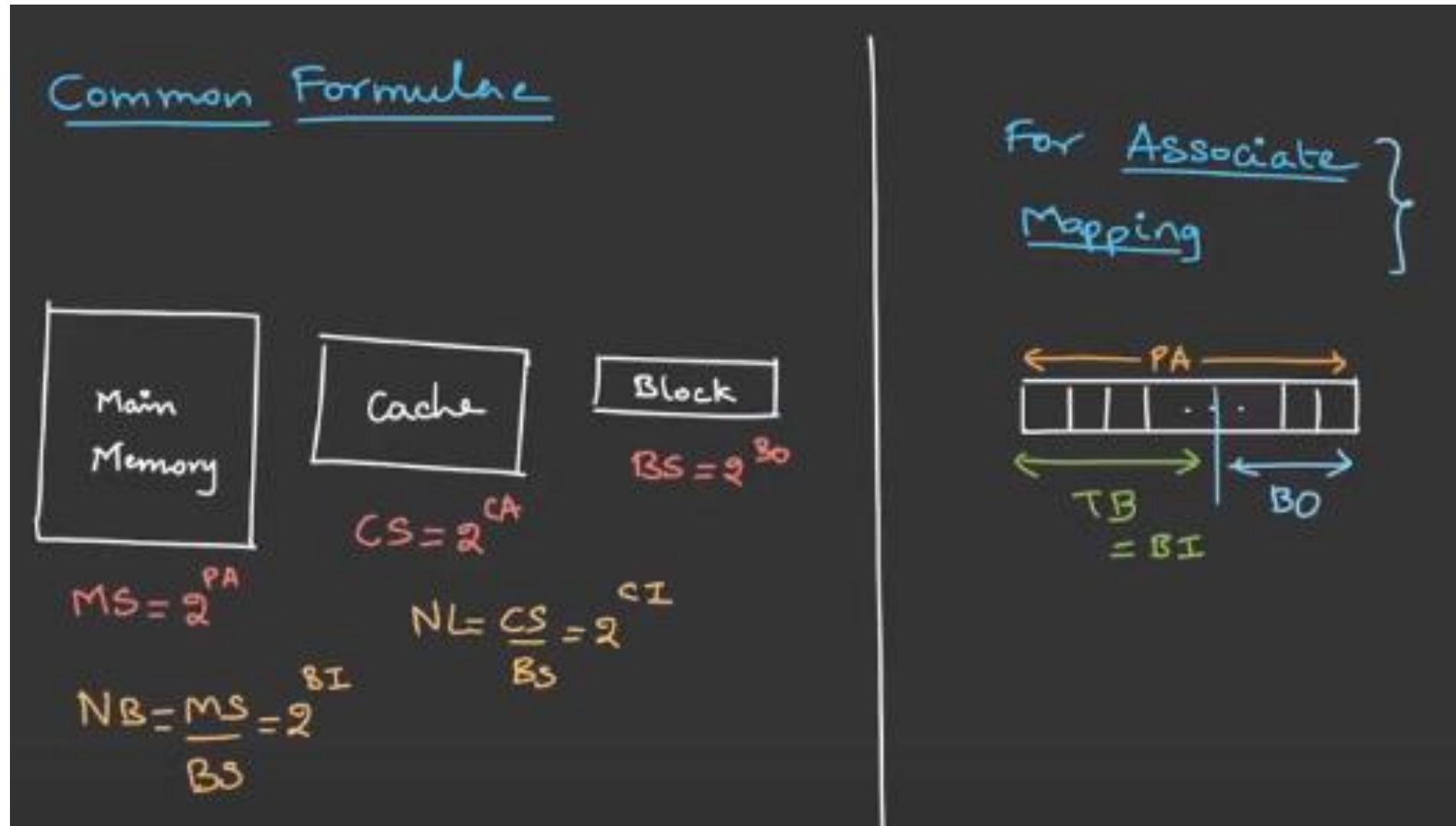
Block offset

Cache Miss

(0000) Block -0

8 th Byte	7 th Byte	6 th Byte	5 th Byte	4 th Byte	3 rd Byte	2 nd Byte	1 st Byte

- CPU has choice to copy the Block 3 to any of the cache line. But cpu should remove any one block . To remove it uses Cache replacement algorithm. i.e FIFO, MRU, LRU or Randomly can choose any cache line to remove the block and load the new one.



Q) An 8kB Associative cache has blocks of size 32 byte.
And main memory size is 1GB. Find the number of tag bits and number of comparators required for search?

$$CS = 8kB = 2^{13} B$$

$\downarrow \quad \downarrow$
 $2^3 \quad 2^{10}$

$$BS = 32 B = 2^5 B \quad BO = 5 \text{ bits}$$

$$MS = 1GB = 2^{30} B \quad PA = 30 \text{ bits}$$



Number of
Comparators

= Number of cache lines

= 256 Comparators



$$NL = \frac{CS}{BS} = \frac{2^{13}}{2^5} = 2^8 \text{ lines} = 256$$

Size of comparator: 25 bit

Problem-02:

Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find- Number of bits in tag & Tag directory size

Solution-Given- Cache memory size = 16 KB

Block size = Frame size = Line size = 256 bytes

Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

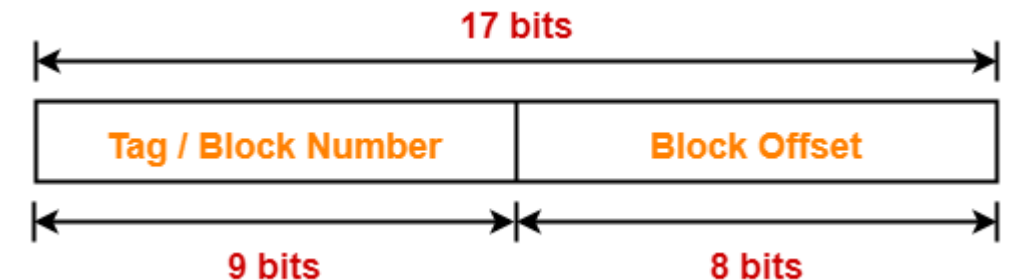
We have, Size of main memory = 128 KB = $2^7 * 2^{10} = 2^{17}$ bytes

Thus, Number of bits in physical address = 17 bits

Number of Bits in Block Offset-

We have, Block size = 256 bytes = 2^8 bytes

Thus, Number of bits in block offset = 8 bits



Number of Bits in Tag-

Number of bits in tag = Number of bits in physical address – Number of bits in block offset

= 17 bits – 8 bits = 9 bits

Thus, Number of bits in tag = 9 bits

Number of Lines in Cache-

Total number of lines in cache
= Cache size / Line size
= 16 KB / 256 bytes
= 2^{14} bytes / 2^8 bytes
= 2^6 lines

Tag Directory Size-

Tag directory size
= Number of tags x Tag size
= Number of lines in cache x Number of bits in tag
= $2^6 \times 9$ bits = 576 bits
= 72 bytes
Thus, size of tag directory = 72 bytes

Problem-03:

Consider a fully associative mapped cache of size 512 KB with block size 1 KB. There are 17 bits in the tag. Find- Size of main memory & Tag directory size

Solution-

Given- Cache memory size = 512 KB
Block size = Frame size = Line size = 1 KB
Number of bits in tag = 17 bits

We consider that the memory is byte addressable.

Number of Bits in Block Offset-

We have, Block size = 1 KB = 2^{10} bytes
Thus, Number of bits in block offset = 10 bits

Number of Bits in Physical Address-

Number of bits in physical address
= Number of bits in tag + Number of bits in block offset
= 17 bits + 10 bits
= 27 bits
Thus, Number of bits in physical address = 27 bits

Size of Main Memory-

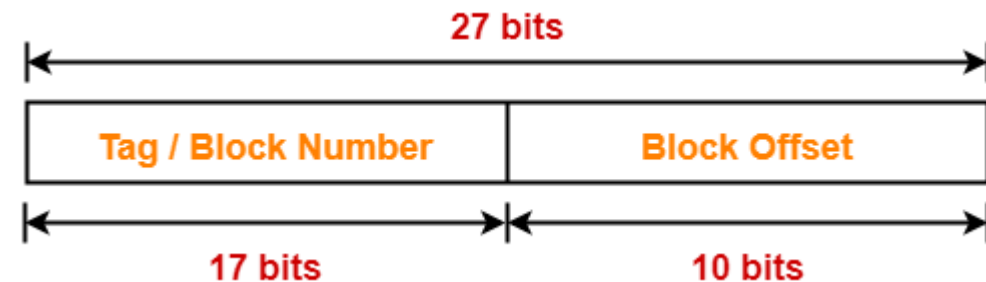
We have, Number of bits in physical address
= 27 bits
Thus, Size of main memory
= 2^{27} bytes = 128 MB

Number of Lines in Cache-

Total number of lines in cache
= Cache size / Line size
= 512 KB / 1 KB
= 512 lines
= 2^9 lines

Tag Directory Size-

Tag directory size = Number of tags x Tag size
= Number of lines in cache x Number of bits in tag
= $2^9 \times 17$ bits
= 8704 bits
= 1088 bytes
Thus, size of tag directory = 1088 bytes



Problem-04:

Consider a fully associative mapped cache with block size 4 KB. The size of main memory is 16 GB. Find the number of bits in tag.

Solution-

Given- Block size = Frame size = Line size = 4 KB

Size of main memory = 16 GB

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

We have, Size of main memory

= 16 GB

= 2^{34} bytes

Thus, Number of bits in physical address = 34 bits

Number of Bits in Block Offset-

We have, Block size

= 4 KB

= 2^{12} bytes

Thus, Number of bits in block offset = 12 bits

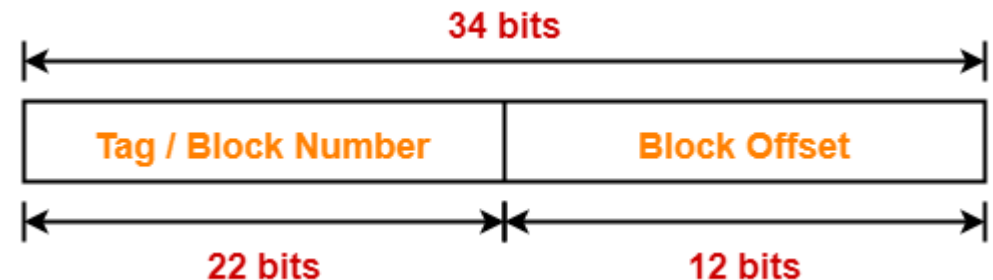
Number of Bits in Tag-

Number of bits in tag =

Number of bits in physical address – Number of bits in block offset

= 34 bits – 12 bits

= 22 bits

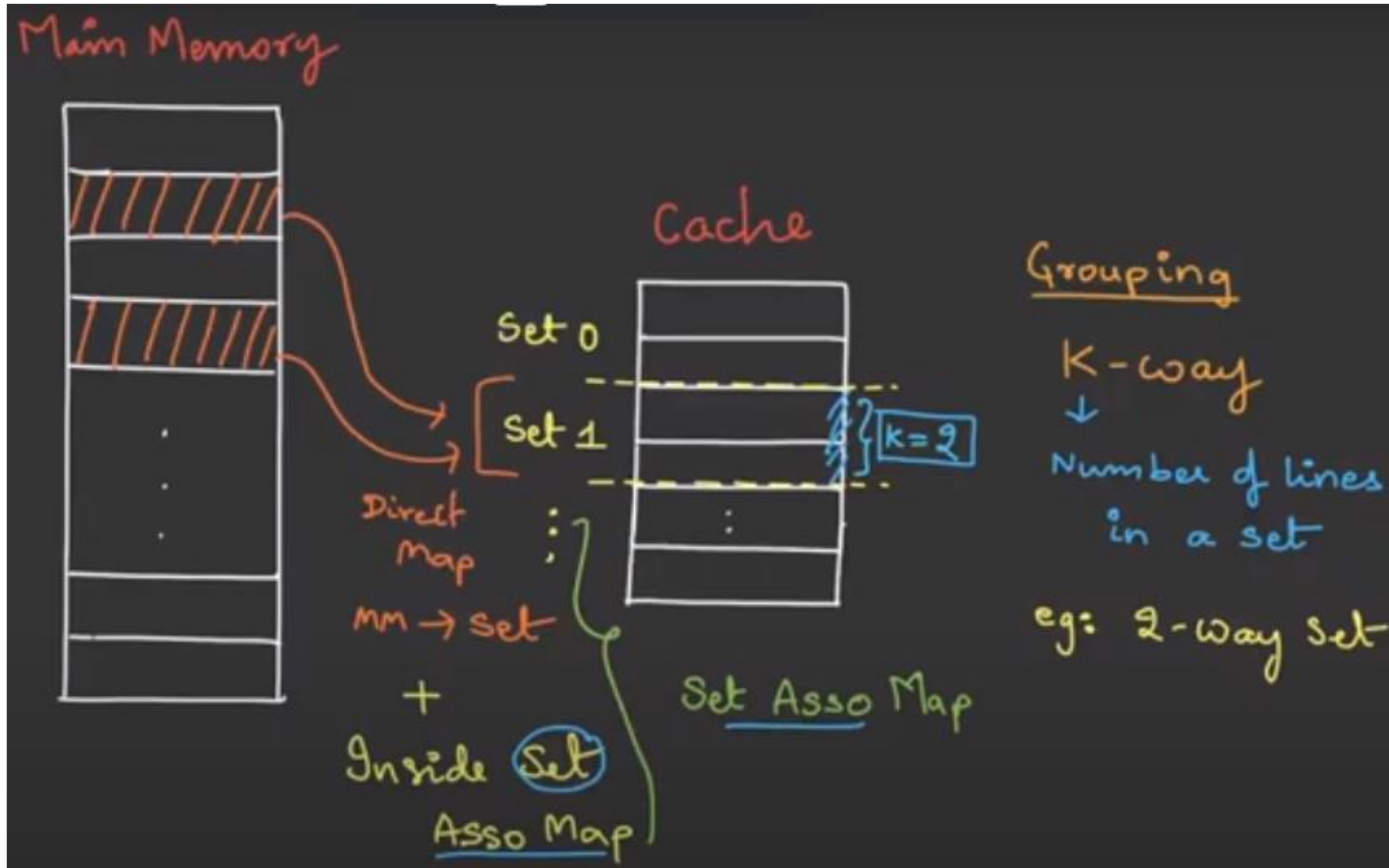


Mapping Functions : K- Way Set Associative Mapping

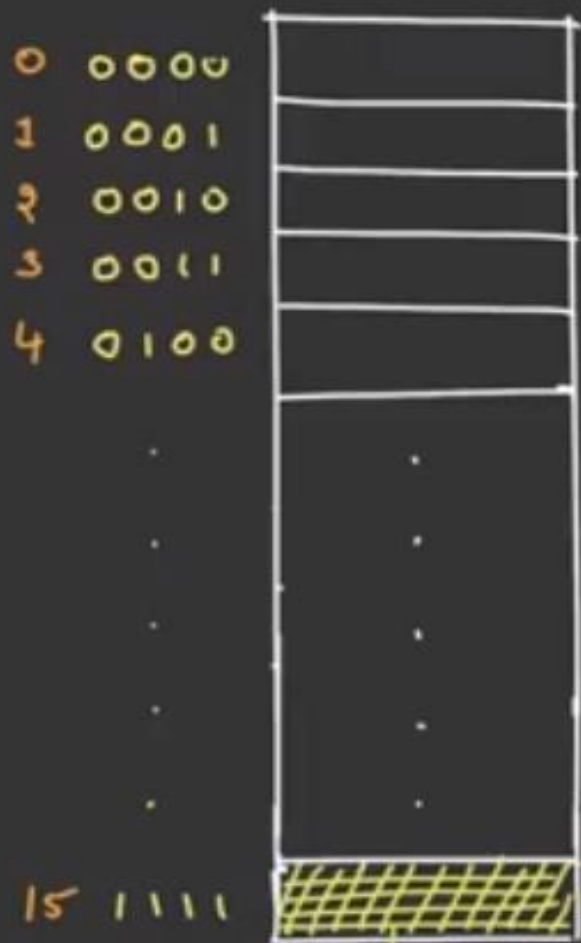
In k-way set associative mapping,

- **Cache lines are grouped into sets** where **each set contains k number of lines**.
- A particular block of main memory can map to only one particular set of the cache (**Direct Mapping**).
- However, within that set, the memory block can map any cache line that is freely available (**Associative Mapping**)
- So Set Associative Mapping is **combination** of Direct mapping and Associative mapping : Direct mapping to identify set and Associative mapping to map within the set
- K represents **numbers lines in a set**
- When a processor generates the **PA** , it uses **SI bits** in PA to identify the **set in Cache memory** , then uses **Tag bits** of CPU address to **compare** with associated Tag bits in tag directory . **If matches Cache hit** is said to occurred.

Mapping Functions : K- Way Set Associative Mapping



Main Memory (128 Bytes)

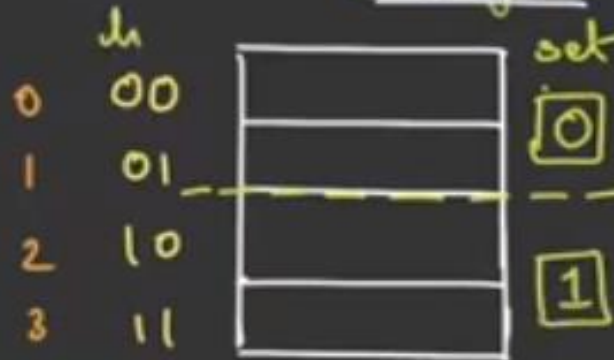


Block (8 bytes)

$$NB = \frac{MS}{BS} = \frac{128}{8} = 16 \text{ blocks}$$

2-way Set Associative Cache
 $K=2$

Cache (32 bytes)



$$NL = \frac{CS}{BS} = \frac{32}{8} = 4 \text{ lines}$$

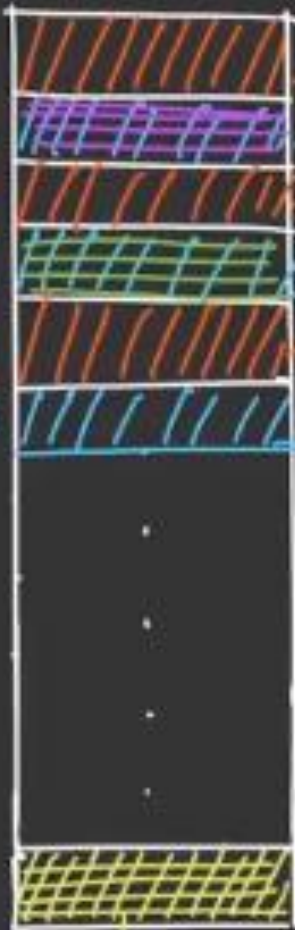
$$NS = \frac{NL}{K} = \frac{4}{2} = 2 \text{ sets} = 2 \text{ sets}$$

Tag Bits



Main Memory (128 Bytes)

0 0000
 1 0001
 2 0010
 3 0011
 4 0100
 5 0101



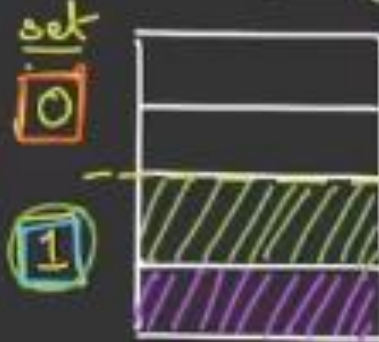
15 1111

Block [8 bytes]

$$NB = \frac{MS}{BS} = \frac{128}{8} = 16 \text{ blocks}$$

2-way Set Associative Cache
 $K=2$

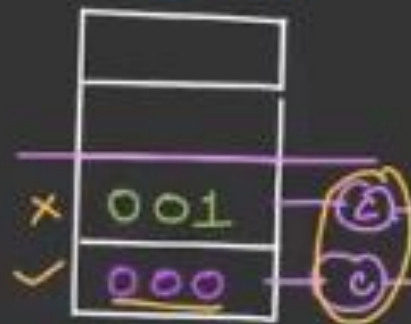
Cache (32 bytes)



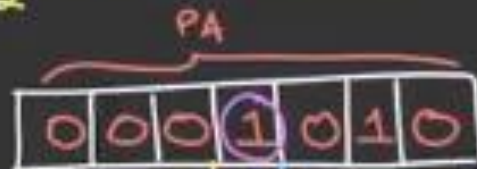
$$NL = \frac{CS}{BS} = \frac{32}{8} = 4 \text{ lines}$$

$$NS = \frac{NL}{K} = \frac{4}{2} = 2 \text{ sets} = 2^1 \text{ sets}$$

Tag Bits

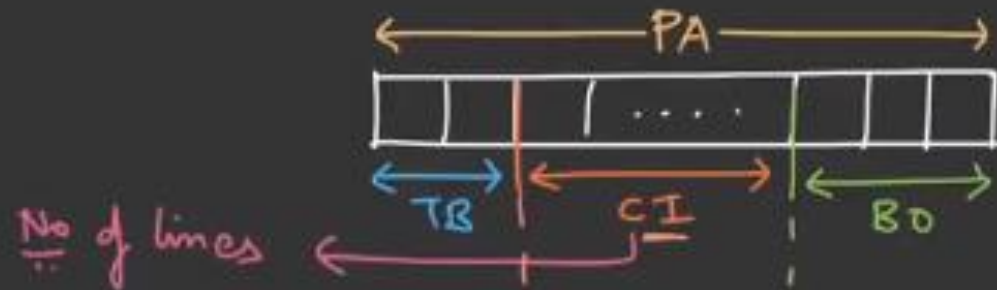


CPU

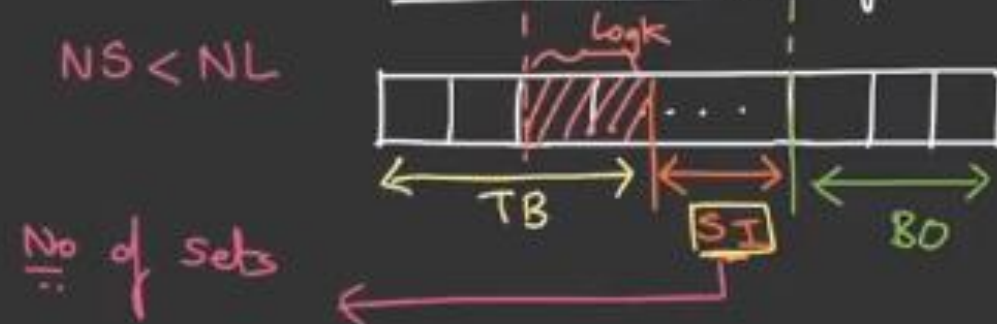


Hit

Direct Mapping



Set Asso Mapping



$$NS < NL$$

$$NL = \frac{CS}{BS} = 2^{CI}$$

$$NS = \frac{NL}{K} = \frac{2^{CI}}{K}$$

$$K = 2^x$$

$$\log_2 K = \log_2 2^x$$

$$\log_2 K = x \log_2 2$$

$$\log_2 K = x$$

$$NS = \frac{2^{CI}}{2^x} = \frac{2^{CI}}{2^{\log K}} = \frac{2^{CI - \log K}}{2^{SI}} = 2$$

$$SI = CI - \log K$$

Common Formulae



$$MS = 2^{PA}$$

$$NB = \frac{MS}{BS} = 2^{SI}$$



$$CS = 2^{CA}$$

$$NL = \frac{CS}{BS} = 2^{CI}$$



$$BS = 2^{BO}$$

$$NS = \frac{NL}{K} = 2^{SI}$$

$$SI = CI - \log K$$

Set Associate Mapping



Problem-01: Consider a 2-way set associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find- Number of bits in tag & Tag directory size

Problem-02: Consider a 8-way set associative mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find- Size of main memory & Tag directory size

Problem-03: Consider a 4-way set associative mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find- Size of cache memory & Tag directory size

Problem-04: Consider a 8-way set associative mapped cache. The size of cache memory is 512 KB and there are 10 bits in the tag. Find the size of main memory.

Problem-05: Consider a 4-way set associative mapped cache. The size of main memory is 64 MB and there are 10 bits in the tag. Find the size of cache memory.

Problem-01:

Consider a 2-way set associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find- Number of bits in tag & Tag directory size

Solution-

Given- Set size = 2

Cache memory size = 16 KB

Block size = Frame size = Line size = 256 bytes

Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

We have,

Size of main memory = 128 KB = 2^{17} bytes

Thus, Number of bits in physical address = 17 bits

Number of Bits in Block Offset-

We have,

Block size = 256 bytes = 2^8 bytes

Thus, Number of bits in block offset = 8 bits

Number of Lines in Cache-

Total number of lines in cache

= Cache size / Line size

= 16 KB / 256 bytes

= 2^{14} bytes / 2^8 bytes

= 64 lines

Thus, Number of lines in cache = 64 lines

Number of Sets in Cache-

Total number of sets in cache

= Total number of lines in cache / Set size

= $64 / 2 = 32$ sets

= 2^5 sets

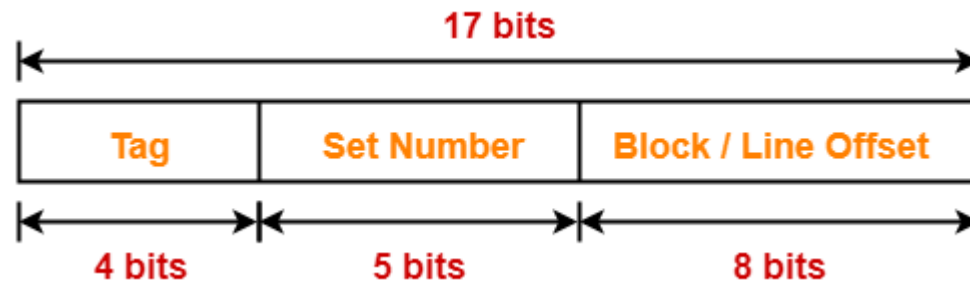
Thus, Number of bits in set number = 5 bits

Number of Bits in Tag-

Number of bits in tag
= Number of bits in physical address – (Number of bits in set number + Number of bits in block offset)
= 17 bits – (5 bits + 8 bits)
= 17 bits – 13 bits
= 4 bits
Thus, Number of bits in tag = 4 bits

Tag Directory Size-

Tag directory size
= Number of tags x Tag size
= Number of lines in cache x Number of bits in tag
= 64 x 4 bits = 256 bits
= 32 bytes
Thus, size of tag directory = 32 bytes



Problem-02:

Consider a 8-way set associative mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find-
Size of main memory & Tag directory size

Solution-

Given- Set size = 8

Cache memory size = 512 KB

Block size = Frame size = Line size = 1 KB

Number of bits in tag = 7 bits

We consider that the memory is byte addressable.

Number of Bits in Block Offset-

We have, Block size = 1 KB

= 2^{10} bytes

Thus, Number of bits in block offset = 10 bits

Number of Lines in Cache-

Total number of lines in cache

= Cache size / Line size

= 512 KB / 1 KB

= 512 lines

Thus, Number of lines in cache = 512 lines

Number of Sets in Cache-

Total number of sets in cache
= Total number of lines in cache / Set size
= $512 / 8$
= 64 sets
= 2^6 sets
Thus, Number of bits in set number = 6 bits

Size of Main Memory-

We have,
Number of bits in physical address = 23 bits
Thus, Size of main memory
= 2^{23} bytes
= 8 MB

Number of Bits in Physical Address-

Number of bits in physical address
= Number of bits in tag + Number of bits in set number +
Number of bits in block offset
= 7 bits + 6 bits + 10 bits
= 23 bits
Thus, Number of bits in physical address = 23 bits

Tag Directory Size-

Tag directory size
= Number of tags x Tag size
= Number of lines in cache x Number of bits in tag
= 512×7 bits
= 3584 bits
= 448 bytes
Thus, size of tag directory = 448 bytes



Problem-03:

Consider a 4-way set associative mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find- Size of cache memory & Tag directory size

Solution-

Given-

Set size = 4

Block size = Frame size = Line size = 4 KB

Main memory size = 16 GB

Number of bits in tag = 10 bits

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

We have,

Size of main memory

= 16 GB

= 2^{34} bytes

Thus, Number of bits in physical address = 34 bits

Number of Bits in Block Offset-

We have,

Block size

= 4 KB

= 2^{12} bytes

Thus, Number of bits in block offset = 12 bits

Number of Bits in Set Number-

Number of bits in set number

= Number of bits in physical address – (Number of bits in tag +

Number of bits in block offset)

= 34 bits – (10 bits + 12 bits)

= 34 bits – 22 bits

= 12 bits

Thus, Number of bits in set number = 12 bits

Number of Lines in Cache-

We have-

Total number of sets in cache = 2^{12} sets

Each set contains 4 lines

Thus,

Total number of lines in cache

= Total number of sets in cache x Number of lines in each set

= 2^{12} x 4 lines

= 2^{14} lines

Number of Sets in Cache-

We have-

Number of bits in set number = 12 bits

Thus, Total number of sets in cache = 2^{12} sets

Size of Cache Memory-

Size of cache memory

= Total number of lines in cache x Line size

= 2^{14} x 4 KB

= 2^{16} KB

= 64 MB

Thus, Size of cache memory = 64 MB

Tag Directory Size-

Tag directory size

= Number of tags x Tag size

= Number of lines in cache x Number of bits in tag

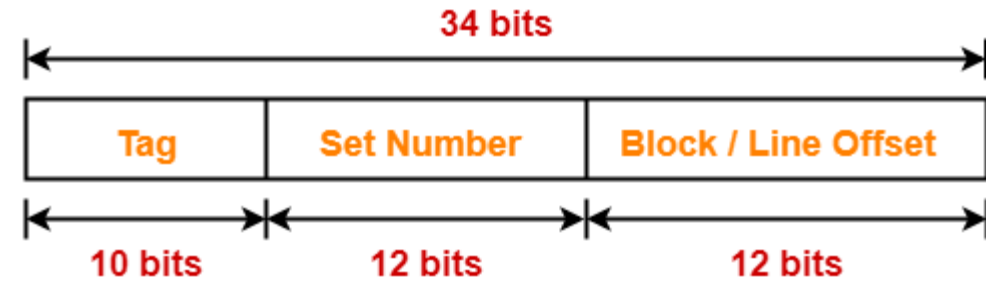
= $2^{14} \times 10$ bits

= 163840 bits

= 20480 bytes

= 20 KB

Thus, size of tag directory = 20 KB



Problem-04:

Consider a 8-way set associative mapped cache. The size of cache memory is 512 KB and there are 10 bits in the tag. Find the size of main memory.

Solution-

Given-

Set size = 8

Cache memory size = 512 KB

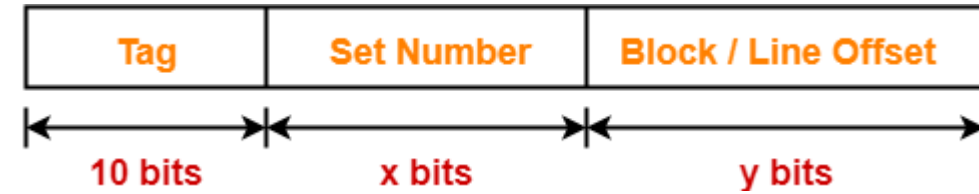
Number of bits in tag = 10 bits

We consider that the memory is byte addressable.

Let-

Number of bits in set number field = x bits

Number of bits in block offset field = y bits



Sum of Number Of Bits Of Set Number Field And Block Offset Field-

We have,

Cache memory size = Number of sets in cache $\times$ Number of lines in one set $\times$ Line size

Now, substituting the values, we get-

$$512 \text{ KB} = 2^x \times 8 \times 2^y \text{ bytes}$$

$$2^{19} \text{ bytes} = 2^{3+x+y} \text{ bytes}$$

$$19 = 3 + x + y$$

$$x + y = 19 - 3$$

$$x + y = 16$$

Number of Bits in Physical Address-

Number of bits in physical address

= Number of bits in tag + Number of bits in set number +
Number of bits in block offset

$$= 10 \text{ bits} + x \text{ bits} + y \text{ bits}$$

$$= 10 \text{ bits} + (x + y) \text{ bits}$$

$$= 10 \text{ bits} + 16 \text{ bits}$$

$$= 26 \text{ bits}$$

Thus, Number of bits in physical address = 26 bits

Size of Main Memory-

We have,

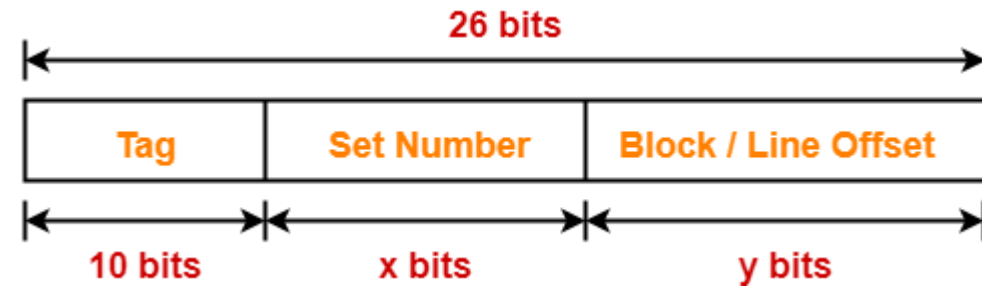
Number of bits in physical address = 26 bits

Thus, Size of main memory

= 2^{26} bytes

= 64 MB

Thus, size of main memory = 64 MB



Problem-05:

Consider a 4-way set associative mapped cache. The size of main memory is 64 MB and there are 10 bits in the tag. Find the size of cache memory.

Solution-

Given-

Set size = 4

Main memory size = 64 MB

Number of bits in tag = 10 bits

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

We have,

Size of main memory

= 64 MB

= 2^{26} bytes

Thus, Number of bits in physical address = 26 bits

Sum Of Number Of Bits Of Set Number Field And Block Offset Field-

Let-

Number of bits in set number field = x bits

Number of bits in block offset field = y bits

Then, Number of bits in physical address

= Number of bits in tag + Number of bits in set number +
Number of bits in block offset

So, we have-

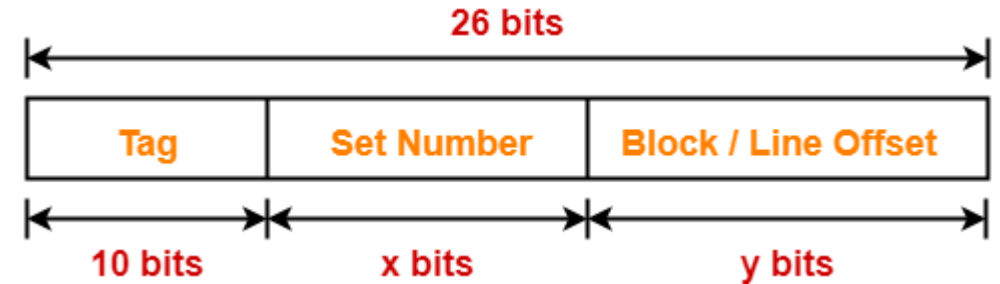
26 bits = 10 bits + x bits + y bits

$26 = 10 + (x + y)$

$x + y = 26 - 10$

$x + y = 16$

Thus, Sum of number of bits of set number field and block offset field = 16 bits



Size of Cache Memory-

Cache memory size

= Number of sets in cache x Number of lines in one set x Line size

= $2^x \times 4 \times 2^y$ bytes

= 2^{2+x+y} bytes

= 2^{2+16} bytes

= 2^{18} bytes

= 256 KB

Thus, size of cache memory = 256 KB

Virtual memories

- Recall that an important challenge in the design of a computer system is to provide a large, fast memory system at an affordable cost.
- Architectural solutions to increase the effective speed and size of the memory system.
- Cache memories were developed to increase the effective speed of the memory system.
- Virtual memory is an architectural solution to increase the effective size of the memory system.

Virtual memories (contd..)

- Recall that the addressable memory space depends on the number of address bits in a computer.
 - For example, if a computer issues 32-bit addresses, the addressable memory space is 4G bytes.
- Physical main memory in a computer is generally not as large as the entire possible addressable space.
 - Physical memory typically ranges from a few hundred megabytes to 1G bytes.
- Large programs that cannot fit completely into the main memory have their parts stored on secondary storage devices such as magnetic disks.
 - Pieces of programs must be transferred to the main memory from secondary storage before they can be executed.

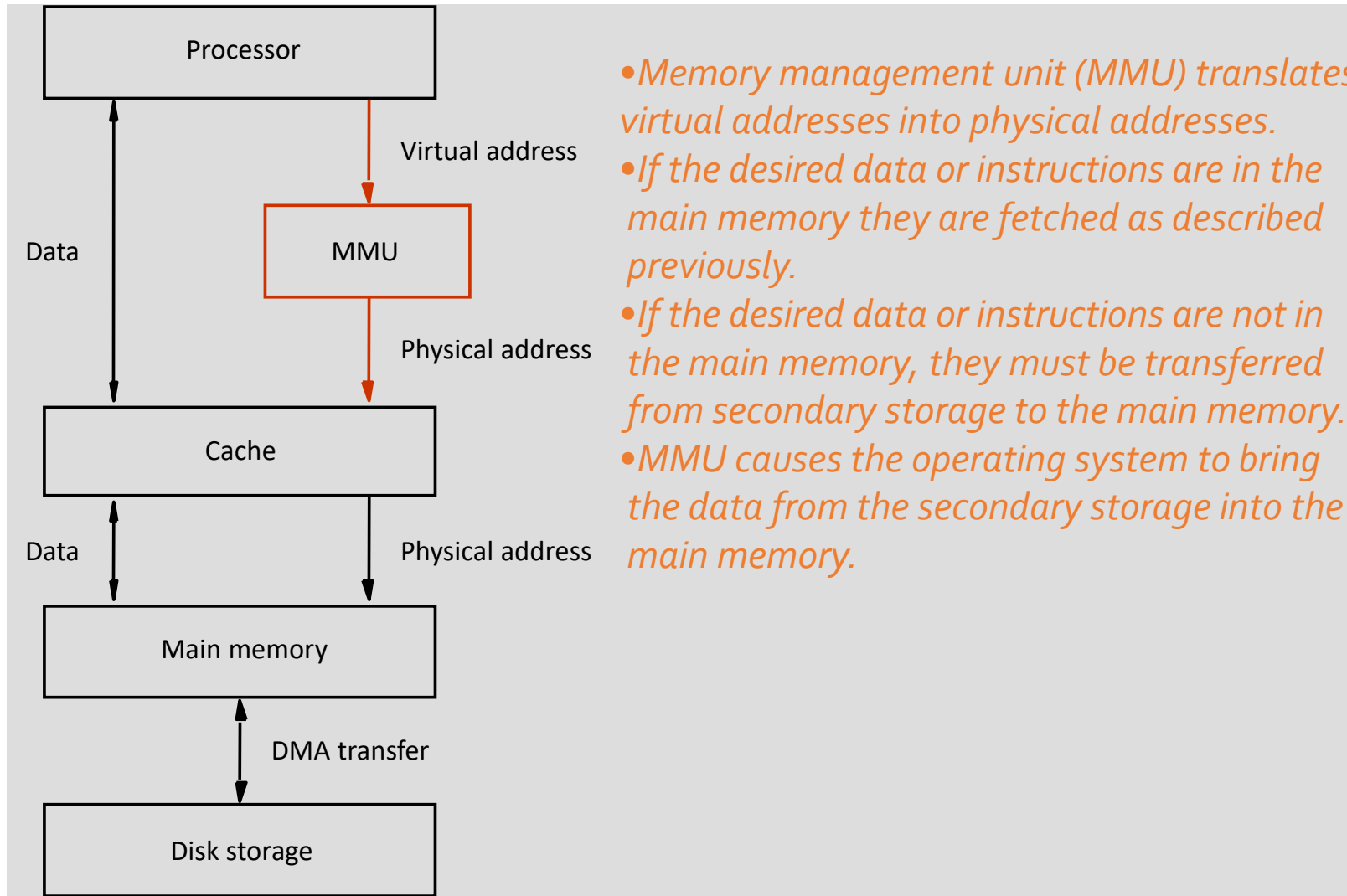
Virtual memories (contd..)

- When a new piece of a program is to be transferred to the main memory, and the main memory is full, then some other piece in the main memory must be replaced.
 - Recall this is very similar to what we studied in case of cache memories.
- Operating system automatically transfers data between the main memory and secondary storage.
 - Application programmer need not be concerned with this transfer.
 - Also, application programmer does not need to be aware of the limitations imposed by the available physical memory.

Virtual memories (contd..)

- Techniques that automatically move program and data between main memory and secondary storage when they are required for execution are called virtual-memory techniques.
- Programs and processors reference an instruction or data independent of the size of the main memory.
- Processor issues binary addresses for instructions and data.
 - These binary addresses are called logical or virtual addresses.
- **Virtual addresses are translated into physical addresses** by a combination of hardware and software subsystems.
 - If virtual address refers to a part of the program that is currently in the main memory, it is accessed immediately.
 - If the address refers to a part of the program that is not currently in the main memory, it is first transferred to the main memory before it can be used.

Virtual memory organization



Address translation

- Assume that program and data are composed of fixed-length units called pages.
- A page consists of a block of words that occupy contiguous locations in the main memory.
- Page is a basic unit of information that is transferred between secondary storage and main memory.
- Size of a page commonly ranges from 2K to 16K bytes.
 - Pages should not be too small, because the access time of a secondary storage device is much larger than the main memory.
 - Pages should not be too large, else a large portion of the page may not be used, and it will occupy valuable space in the main memory.

Address translation (contd..)

- Concepts of virtual memory are similar to the concepts of cache memory.
- Cache memory:
 - Introduced to bridge the speed gap between the processor and the main memory.
 - Implemented in hardware.
- Virtual memory:
 - Introduced to bridge the speed gap between the main memory and secondary storage.
 - Implemented in part by software.

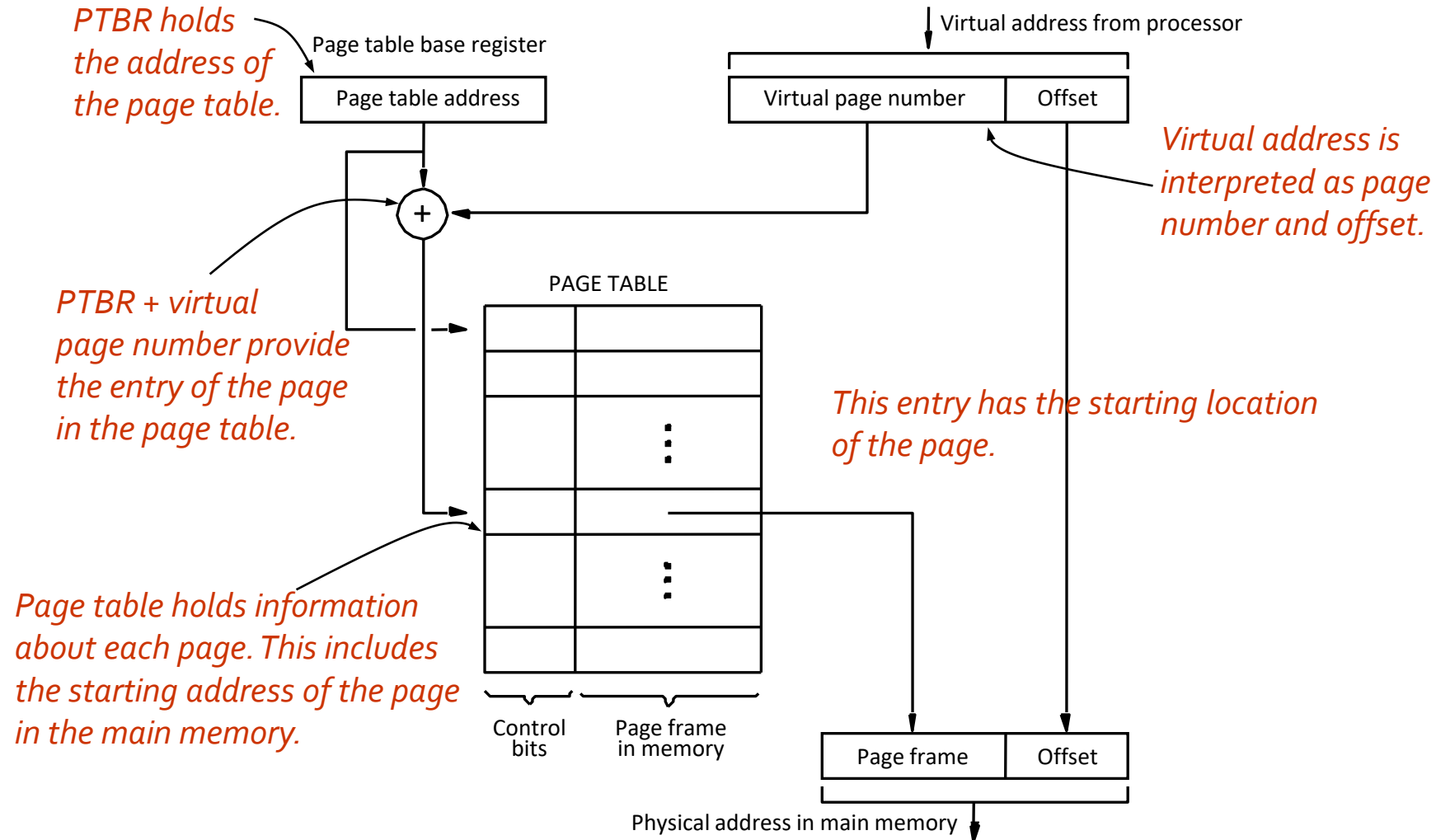
Address translation (contd..)

- Each virtual or logical address generated by a processor is interpreted as a virtual page number (high-order bits) plus an offset (low-order bits) that specifies the location of a particular byte within that page.
- Information about the main memory location of each page is kept in the page table.
 - Main memory address where the page is stored.
 - Current status of the page.
- Area of the main memory that can hold a page is called as page frame.
- Starting address of the page table is kept in a page table base register.

Address translation (contd..)

- Virtual page number generated by the processor is added to the contents of the page table base register.
 - This provides the address of the corresponding entry in the page table.
- The contents of this location in the page table give the starting address of the page if the page is currently in the main memory.

Address translation (contd..)



Address translation (contd..)

- Page table entry for a page also includes some control bits which describe the status of the page while it is in the main memory.
- One bit indicates the validity of the page.
 - Indicates whether the page is actually loaded into the main memory.
 - Allows the operating system to invalidate the page without actually removing it.
- One bit indicates whether the page has been modified during its residency in the main memory.
 - This bit determines whether the page should be written back to the disk when it is removed from the main memory.
 - Similar to the dirty or modified bit in case of cache memory.

Address translation (contd..)

- Other control bits for various other types of restrictions that may be imposed.
 - For example, a program may only have read permission for a page, but not write or modify permissions.

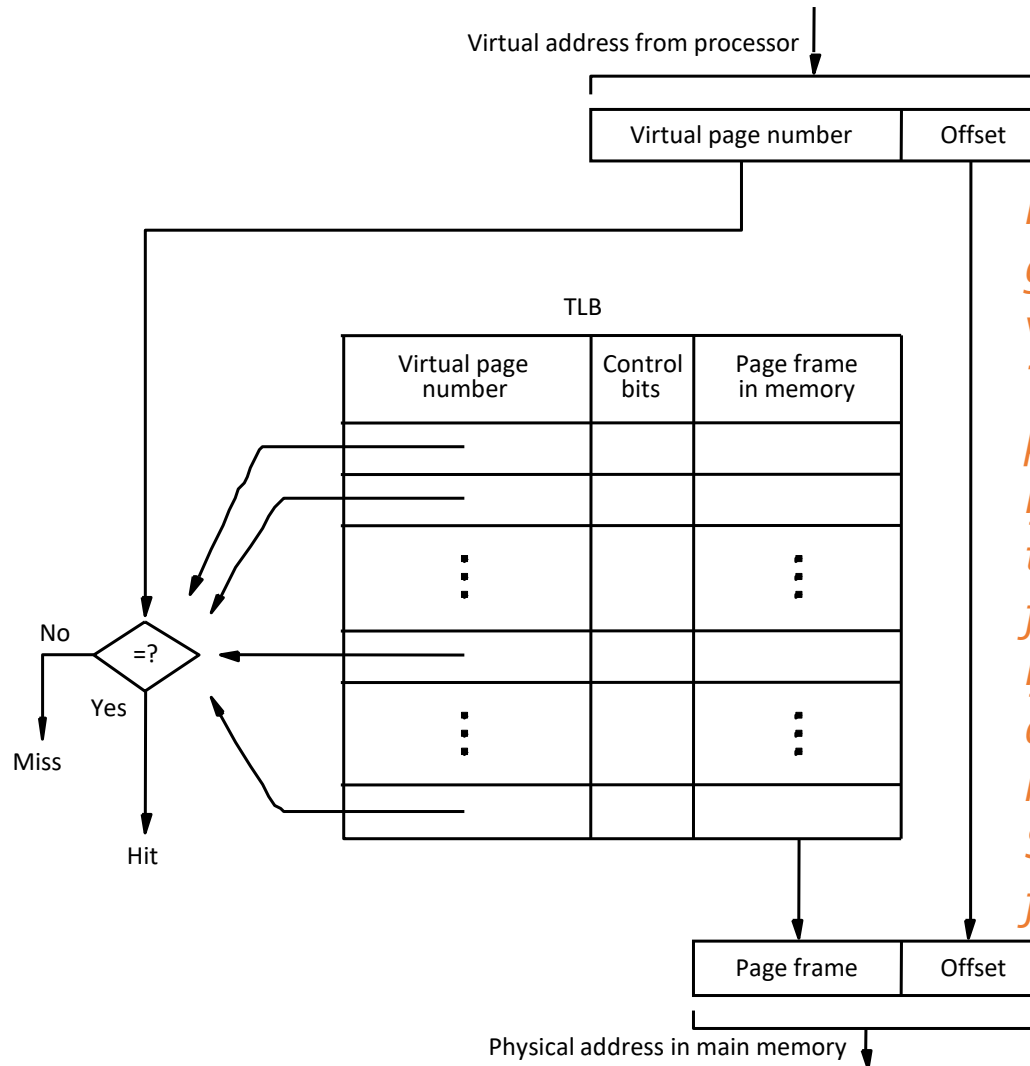
Address translation (contd..)

- Where should the page table be located?
- Recall that the page table is used by the MMU for every read and write access to the memory.
 - Ideal location for the page table is within the MMU.
- Page table is quite large.
- MMU is implemented as part of the processor chip.
- Impossible to include a complete page table on the chip.
- Page table is kept in the main memory.
- A copy of a small portion of the page table can be accommodated within the MMU.
 - Portion consists of page table entries that correspond to the most recently accessed pages.

Address translation (contd..)

- A small cache called as Translation Lookaside Buffer (TLB) is included in the MMU.
 - TLB holds page table entries of the most recently accessed pages.
- Recall that cache memory holds most recently accessed blocks from the main memory.
 - Operation of the TLB and page table in the main memory is similar to the operation of the cache and main memory.
- Page table entry for a page includes:
 - Address of the page frame where the page resides in the main memory.
 - Some control bits.
- In addition to the above for each page, TLB must hold the virtual page number for each page.

Address translation (contd..)



Associative-mapped TLB

High-order bits of the virtual address generated by the processor select the virtual page.

These bits are compared to the virtual page numbers in the TLB.

If there is a match, a hit occurs and the corresponding address of the page frame is read.

If there is no match, a miss occurs and the page table within the main memory must be consulted.

Set-associative mapped TLBs are found in commercial processors.

Address translation (contd..)

- How to keep the entries of the TLB coherent with the contents of the page table in the main memory?
- Operating system may change the contents of the page table in the main memory.
 - Simultaneously it must also invalidate the corresponding entries in the TLB.
- A control bit is provided in the TLB to invalidate an entry.
- If an entry is invalidated, then the TLB gets the information for that entry from the page table.
 - Follows the same process that it would follow if the entry is not found in the TLB or if a “miss” occurs.

Address translation (contd..)

- What happens if a program generates an access to a page that is not in the main memory?
- In this case, a page fault is said to occur.
 - Whole page must be brought into the main memory from the disk, before the execution can proceed.
- Upon detecting a page fault by the MMU, following actions occur:
 - MMU asks the operating system to intervene by raising an exception.
 - Processing of the active task which caused the page fault is interrupted.
 - Control is transferred to the operating system.
 - Operating system copies the requested page from secondary storage to the main memory.
 - Once the page is copied, control is returned to the task which was interrupted.

Address translation (contd..)

- Servicing of a page fault requires transferring the requested page from secondary storage to the main memory.
- This transfer may incur a long delay.
- While the page is being transferred the operating system may:
 - Suspend the execution of the task that caused the page fault.
 - Begin execution of another task whose pages are in the main memory.
- Enables efficient use of the processor.

Address translation (contd..)

- How to ensure that the interrupted task can continue correctly when it resumes execution?
- There are two possibilities:
 - Execution of the interrupted task must continue from the point where it was interrupted.
 - The instruction must be restarted.
- Which specific option is followed depends on the design of the processor.

Address translation (contd..)

- How to ensure that the interrupted task can continue correctly when it resumes execution?
- There are two possibilities:
 - Execution of the interrupted task must continue from the point where it was interrupted.
 - The instruction must be restarted.
- Which specific option is followed depends on the design of the processor.

Address translation (contd..)

- When a new page is to be brought into the main memory from secondary storage, the main memory may be full.
 - Some page from the main memory must be replaced with this new page.
- How to choose which page to replace?
 - This is similar to the replacement that occurs when the cache is full.
 - The principle of locality of reference (?) can also be applied here.
 - A replacement strategy similar to LRU can be applied.
- Since the size of the main memory is relatively larger compared to cache, a relatively large amount of programs and data can be held in the main memory.
 - Minimizes the frequency of transfers between secondary storage and main memory.

Address translation (contd..)

- A page may be modified during its residency in the main memory.
- When should the page be written back to the secondary storage?
- Recall that we encountered a similar problem in the context of cache and main memory:
 - Write-through protocol(?)
 - Write-back protocol(?)
- Write-through protocol cannot be used, since it will incur a long delay each time a small amount of data is written to the disk.